Low-Level Design Report

sudo

Başak Güney

Ecesu Bozkurt

İnci Sıla Kaleli

Table of Contents

1. Introduction

Artificial intelligence has become increasingly integrated into recruitment and assessment processes. However, many new graduates and early-career candidates lack structured opportunities to practice interviews under realistic yet supportive conditions. Traditional preparation methods often fail to provide measurable, objective, and personalized feedback regarding both content quality and communication behavior.

The purpose of this system is to provide a web-based AI Mock Interview platform that simulates short (10–15 minute) HR or Technical (non-coding) interviews through a voice-based AI interviewer. The platform enables candidates to configure interview parameters including interview type, target role, company context, domain of interest, difficulty level, and interaction mode (Supportive or Neutral). During the session, the system manages conversational flow, applies rule-based control constraints for time enforcement and scope boundaries, and adapts interviewer behavior according to the selected interaction mode.

A central component of the system is its evidence-based feedback mechanism. At the end of each session, the platform generates a structured feedback report that integrates content-level evaluation (relevance, clarity, completeness) with measurable communication indicators such as pause frequency, filler word usage, speech pace, head movement patterns, and focus/engagement proxies derived from camera analysis (subject to explicit user consent). In Supportive Mode, additional behavioral policies are applied to gently redirect off-topic responses and provide clarifying assistance when candidates express uncertainty.

This Low-Level Design (LLD) document describes the internal structure, class-level responsibilities, interface contracts, control logic, and implementation-oriented design decisions of the AI-Powered Mock Interview Platform. The document elaborates on object design trade-offs, interface documentation guidelines, engineering standards, and key terminology used throughout the implementation.

1.1. Object design trade-offs

1.1.1. Flexibility vs. Control (LLM-Driven Dialogue vs. Rule-Based Control Constraints)

One of the primary design decisions concerned how much control should be delegated to the Large Language Model (LLM) during the interview session.

A fully LLM-driven system would provide high conversational flexibility and natural dialogue progression. However, relying entirely on the LLM would reduce predictability and make it difficult to enforce difficulty boundaries, time constraints, and consistent interaction behavior across different sessions.

To balance flexibility and reliability, a hybrid architecture was adopted. The LLM is responsible for generating interview questions, follow-up prompts, and content-level

evaluations. In contrast, a Backend Orchestrator applies rule-based control constraints that regulate session timing, enforce scope limitations, and maintain interaction consistency according to the selected mode.

This approach improves testability, transparency, and system stability while preserving conversational naturalness.

1.1.2. Modularity vs. Simplicity (Separation of Behavioral Analysis)

Another key decision involved whether behavioral signal extraction should be embedded directly into the dialogue generation logic or implemented as a separate analytical component.

Embedding behavioral analysis inside LLM prompts would simplify integration but would reduce explainability and make the scoring process less transparent. Additionally, it would tightly couple external AI behavior with internal evaluation logic.

Instead, the system separates behavioral analysis into a dedicated BehaviorAnalyzer module. This module processes speech and camera-derived signals independently and produces normalized metrics that are later aggregated by the FeedbackAggregator.

Although this modular design introduces additional structural complexity, it improves maintainability, explainability, and supports graceful degradation if certain analysis components fail.

1.1.3. Accuracy vs. Efficiency (Behavioral Signal Processing)

Behavioral analysis can involve computationally intensive techniques such as deep emotion recognition or advanced prosody modeling. However, the project scope and real-time constraints require efficient processing.

Therefore, the system prioritizes interpretable and computationally lightweight indicators, including:

- Pause duration and frequency
- Filler word occurrence
- Speaking pace estimation
- Head movement patterns
- Focus/engagement proxies

This design favors real-time responsiveness and explainability over highly complex but opaque models.

1.1.4. Security vs. Usability (Consent and Data Handling)

The platform requires access to sensitive resources such as microphone and camera input. While minimizing user friction is important for usability, privacy and legal compliance take priority.

To ensure secure operation:

- Microphone and camera consent is mandatory before session initiation.
- API credentials are stored exclusively on the backend.
- Raw audio and video data are not persistently stored by default.

Although this may slightly increase the number of user interactions, it ensures alignment with privacy-by-design principles and regulatory requirements.

1.1.5. Extensibility vs. Immediate Scope (Optional Features)

The system architecture anticipates optional extensions such as login functionality, session history tracking, CV-based personalization, and avatar-based interviewer visualization.

Instead of tightly coupling these features to the core interview flow, the design isolates them behind modular interfaces. This allows future extensibility without compromising the MVP's stability and timeline constraints.

This trade-off ensures that the core interview loop remains robust while leaving room for incremental system evolution.

1.2. Interface documentation guidelines

In this report, all classes, attributes, and methods follow consistent naming conventions to ensure clarity and maintainability.

Class names are written in PascalCase (e.g., `InterviewSession`, `BehaviorAnalyzer`).
Attributes and method names are written in camelCase (e.g., `sessionId`, `generateQuestion()`).

Class interface descriptions are presented in the following format:

Class: Identifies the name of the software component. Each class represents a distinct responsibility within the system architecture and encapsulates related data and behavior.

Description: Briefly explains the responsibility and scope of the class/component, focusing on what it represents in the system and which concerns it encapsulates.

Attributes: Lists the data members maintained by the class. Attributes define the internal state of the component and represent the information required to fulfill its responsibility. Each attribute is documented with its visibility, name, and data type.

Visibility	Name	Type
public, private, protected	attributeName	String, int, double, boolean, DateTime, Enum, List<Type>

Methods: Defines the behaviors exposed by the class. Methods represent the operations that can be invoked by other components to interact with the class.

Visibility	Signature	Return Type
public, private, protected	methodName(parameterType parameterName, …)	void, String, int, double, boolean, EnumType, List<Type>

1.3. Engineering standards

This Low-Level Design document follows IEEE-style documentation principles for software design reports. Requirement references align with ISO/IEC/IEEE 29148 guidelines and are explicitly mapped in Appendix A (HLD–LLD Traceability Matrix).

The system structure, class relationships, sequence interactions, and state transitions are modeled using Unified Modeling Language (UML) conventions. UML is used to represent:

- Class diagrams (class responsibilities, attributes, and method interfaces)
- Sequence diagram (runtime interaction flow)
- State diagram (InterviewSession lifecycle management)

IEEE Documentation Standards
This report follows IEEE-style documentation principles. Design decisions are justified through explicit trade-off discussions, interfaces and class specifications are described in a structured format, and citations follow IEEE referencing conventions where applicable.

1.4. Definitions, acronyms, and abbreviations

- AI (Artificial Intelligence): Computer systems capable of performing tasks that typically

require human intelligence.

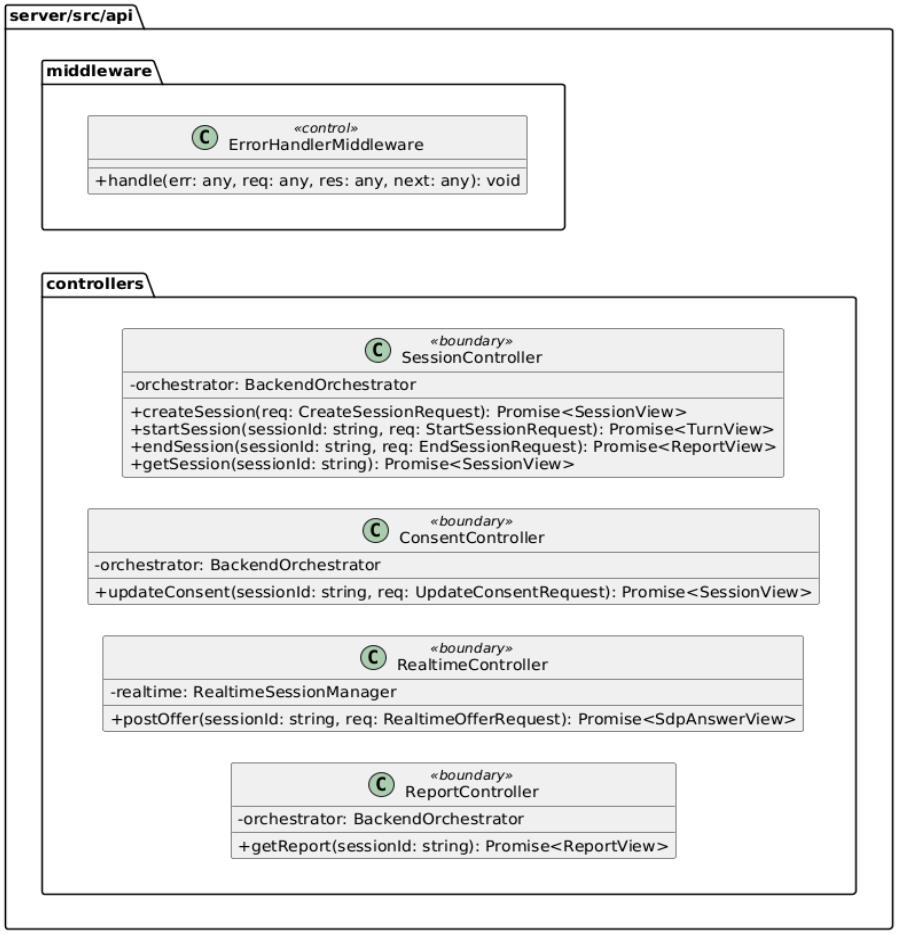
- LLM (Large Language Model): A deep learning–based language model used for natural language generation and evaluation.
- API (Application Programming Interface): A set of endpoints that enables communication between client and server components.
- DTO (Data Transfer Object): A structured object used to transfer data between application layers.
- WebRTC (Web Real-Time Communication): Technology enabling real-time audio/video exchange in web applications.
- SDP (Session Description Protocol): A format used to negotiate media session parameters in WebRTC.
- VAD (Voice Activity Detection): Mechanism for detecting speech activity in real-time audio streams.
- MVP (Minimum Viable Product): The first functional version containing core features.
- Consent: Explicit user permission for microphone and camera access.
- SessionState: Lifecycle state of an interview session (Draft, Configured, ConsentGranted, Ready, InProgress, Ending, ReportReady, Completed, Aborted).
- Intervention: Guardrail-driven action such as time-limit prompting or supportive redirection.
- EvidenceItem: Structured reference linking feedback observations to timestamps or claims.
- Realtime: Live interview flow established via WebRTC and OpenAI Realtime API.

All acronyms are used consistently according to these definitions throughout the document.

2. Packages

2.1. Server

2.1.1. API



2.1.1.1. Controllers

Purpose: Exposes REST endpoints to the client and routes requests into the application layer. It converts request payloads into DTOs, calls orchestration/services, and returns response DTOs. It should stay thin and avoid business logic.

SessionController: Handles session lifecycle endpoints such as creating, starting, ending, and retrieving sessions. It delegates all real business decisions to the orchestrator. It returns client-friendly views of the session and turn results. It is the main entry point for session-related UI actions.

ConsentController: Handles updates related to microphone and camera consent. It ensures consent changes are processed consistently through the orchestrator. It exists to keep consent logic separate from general setup actions. It supports strict “must-have-consent-before-start” workflows.

RealtimeController: Handles the realtime negotiation entry point from the client. It forwards the client’s offer to the realtime manager and returns the SDP answer. It keeps

WebRTC signaling concerns isolated from session lifecycle endpoints. This controller is focused on realtime connectivity rather than interview logic.

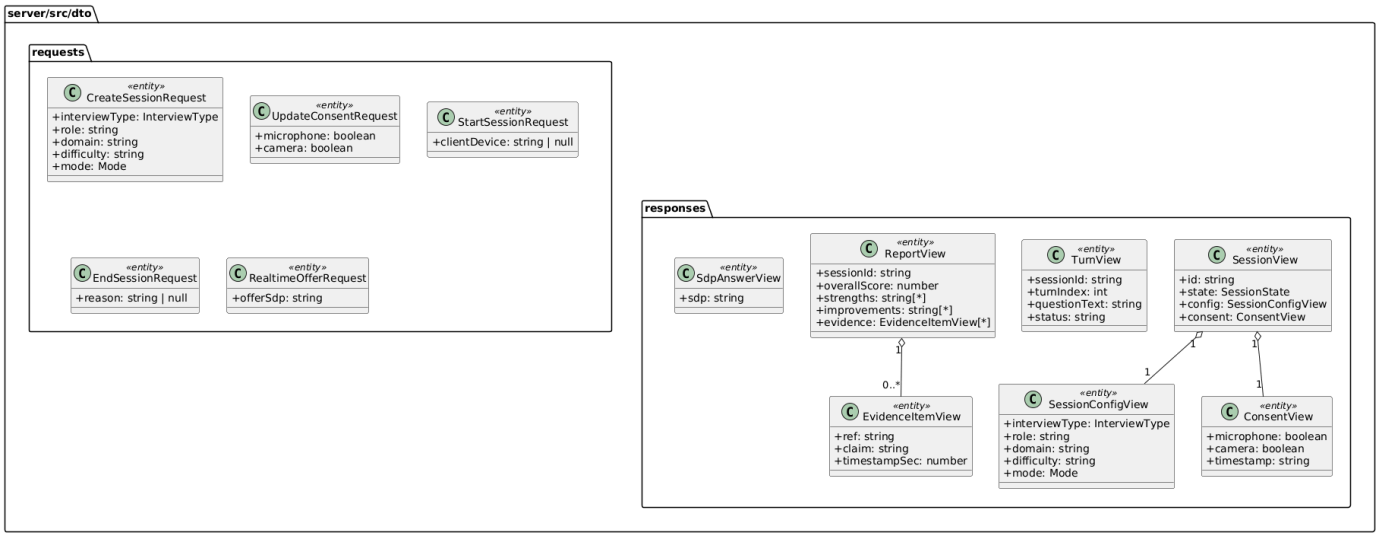
ReportController: Serves the feedback report to the client once the interview is finished. It calls orchestration to retrieve (or produce) the report representation. It keeps reporting concerns separate from session start/end endpoints. This makes reporting easier to evolve without affecting the session flow.

2.1.1.2. Middleware

Purpose: Implements cross-cutting HTTP concerns that apply to many endpoints. It keeps controllers clean by centralizing behaviors like error normalization.

ErrorHandlerMiddleware: Catches errors thrown during request handling and converts them into consistent API responses. It prevents controllers from duplicating try/catch patterns everywhere. It also standardizes error status codes and message formatting. This improves debuggability and client-side reliability.

2.1.2. DTO



2.1.2.1. Requests

Purpose: Defines the input contracts expected from the client. These DTOs represent the JSON payloads for each API action and keep controller signatures consistent and versionable.

CreateSessionRequest: Represents the data required to create a new interview session. It gathers the configuration the AI and session flow will use. It is designed to be stable so

frontend and backend agree on a single contract. It avoids scattering many parameters across endpoints.

UpdateConsentRequest: Represents the user's consent choices for camera and microphone. It supports strict gating of the interview start until consent is granted. It keeps consent updates explicit and auditable. It prevents consent from being mixed into unrelated request payloads.

StartSessionRequest: Represents the payload used when starting a session. It can carry optional metadata that is relevant at start time. It keeps "start" responsibilities separate from "create" responsibilities. It allows the start action to evolve without changing creation DTOs.

EndSessionRequest: Represents the payload used to end a session. It may include a reason or end-context from the client. It supports clean termination flows and future analytics. It keeps end-of-session behavior consistent across clients.

RealtimeOfferRequest: Represents the client's realtime SDP offer. It isolates realtime negotiation inputs from other session endpoints. It makes validation and future extension straightforward. It keeps the controller interface clean and predictable.

2.1.2.2. Responses

Purpose: Defines the output contracts returned to the client. These DTOs are shaped for UI consumption and avoid exposing internal domain structure unnecessarily.

SessionView: A client-facing snapshot of the session state. It provides the minimal session information the UI needs to render the current step. It groups session identifiers, config, consent, and lifecycle state. It supports consistent UI refresh after any session update.

TurnView: A client-facing representation of the current interview turn. It provides the question and status information needed for the interview screen. It is designed to be simple and stable across iterations. It helps the UI display the ongoing interview flow.

ReportView: The client-facing feedback report object. It is shaped for dashboard presentation, including summary fields and evidence references. It abstracts away how the report was produced internally. It is intended as the main output artifact of the system.

SdpAnswerView: Contains the realtime SDP answer returned by the server. It is the minimal structure needed by the client to finalize WebRTC negotiation. It keeps the realtime controller's response contract clear. It supports a clean offer/answer exchange.

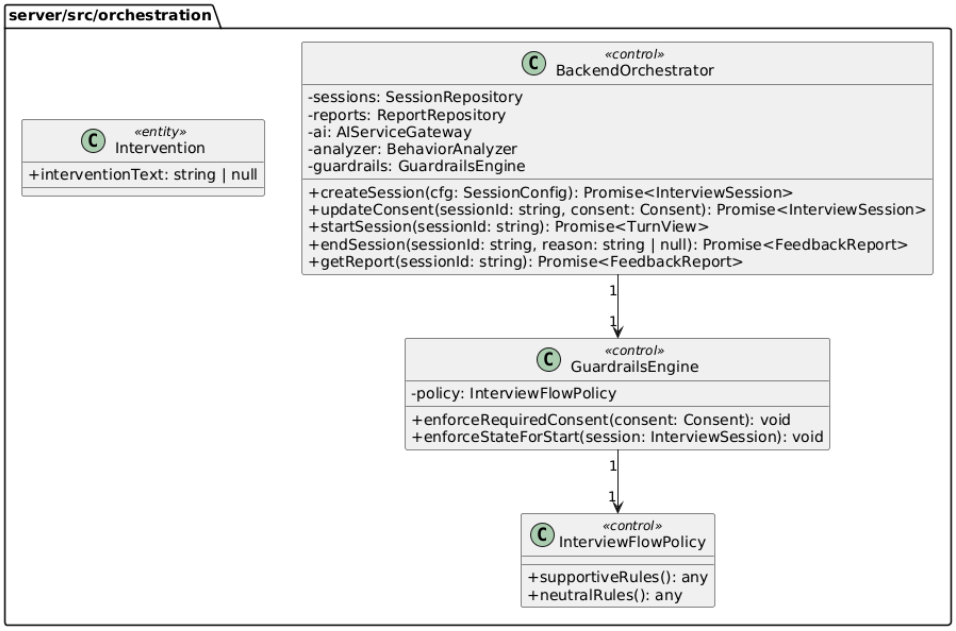
SessionConfigView: A response-friendly representation of the session configuration. It ensures the frontend receives a consistent config format even if internal models change. It keeps the response payload self-contained. It supports UI rendering of "what settings are

active.”

ConsentView: A response-friendly representation of consent. It provides the UI with consent status and a display-ready timestamp. It avoids leaking internal time formats into the client. It supports consistent consent UI indicators.

EvidenceItemView: A response-friendly representation of a single evidence item. It allows the UI to show “why” a score was given with references. It keeps evidence structured and searchable. It improves explainability for feedback.

2.1.3. Orchestration



Purpose: Implements the application/use-case layer. It coordinates workflows, enforces guardrails, updates session state, and calls services and repositories.

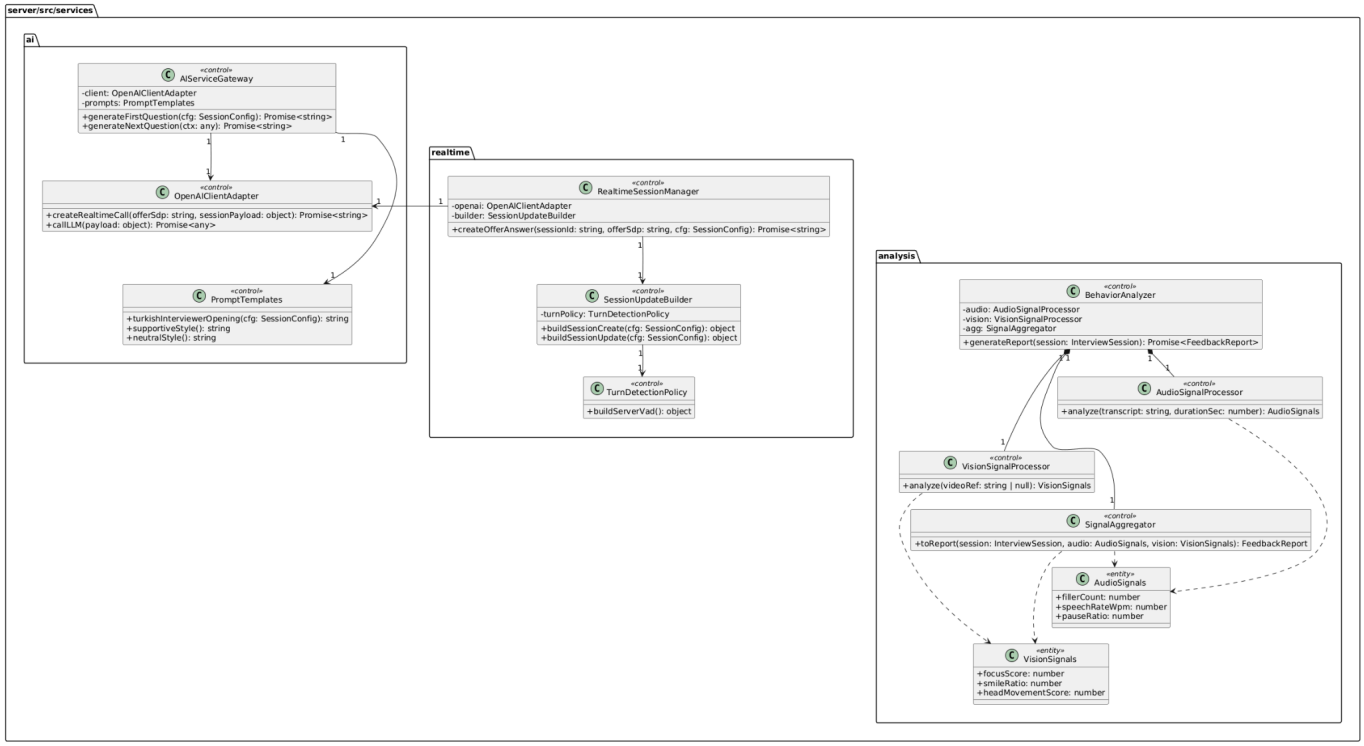
BackendOrchestrator: Coordinates the full interview lifecycle from setup to report generation. It ensures each use-case runs in the correct order and state. It calls repositories for persistence and services for AI and analysis. It keeps controllers thin and centralizes business flow decisions.

GuardrailsEngine: Enforces required constraints such as mandatory camera/microphone consent and valid state transitions. It centralizes policy checks so rules are not duplicated across endpoints. It prevents invalid flows from progressing. It supports consistent behavior across different controllers.

InterviewFlowPolicy: Encapsulates mode-dependent rules for supportive and neutral behavior. It keeps policy definitions separate from enforcement logic. It enables easy updates to interviewing style without rewriting orchestration code. It acts as a single place to define “how this mode should behave.”

Intervention: Represents a guardrail-driven intervention output such as a warning or corrective message. It provides a structured way to express actions that occur when rules are violated. It can be extended later for richer moderation or coaching. In MVP it can remain lightweight.

2.1.4. Services



2.1.4.1. AI

Purpose: Provides AI capabilities and isolates the backend from provider-specific API details. It generates questions/openings and manages prompt consistency across modes.

AIServiceGateway: Offers high-level AI operations such as generating the first and next questions. It ensures interviewer behavior stays consistent with the selected mode. It keeps orchestration independent of the underlying provider call format. It is the main AI entry point for the application flow.

OpenAIClientAdapter: Encapsulates the provider communication details and provides a stable internal interface. It hides authentication, request formatting, and response parsing

concerns. It supports both standard LLM calls and realtime-related calls. It reduces breakage when provider APIs evolve.

PromptTemplates: Stores reusable prompt text and style templates for modes and language. It ensures the interviewer consistently speaks Turkish as required. It centralizes tone rules for supportive vs neutral behavior. It keeps prompt composition maintainable and testable.

2.1.4.2. Realtime

Purpose: Supports realtime interview connectivity and session configuration building. It isolates the realtime negotiation flow from normal REST session management.

RealtimeSessionManager: Coordinates the realtime offer/answer exchange and prepares the server-side configuration used for the realtime session. It acts as the main entry point for realtime negotiation logic. It relies on the adapter to communicate with the external provider. It returns the final SDP answer that the client needs.

SessionUpdateBuilder: Builds provider-specific session payload objects needed for realtime initialization or updates. It keeps payload construction consistent and reusable. It prevents the session manager from becoming bloated with configuration-building logic. It is the single place to update provider schema mapping.

TurnDetectionPolicy: Encapsulates turn-taking configuration such as server-side VAD. It makes turn detection rules configurable without touching higher-level logic. It reduces risk when provider parameters change. It helps keep realtime configuration clean and modular.

2.1.4.3. Analysis

Purpose: Produces the post-session feedback report by extracting signals from audio/text and video and aggregating them into structured feedback.

BehaviorAnalyzer: Orchestrates the overall analysis workflow and produces the final report. It calls the audio and vision processors and then uses the aggregator to build the report. It keeps analysis modular and testable. It is the primary analysis entry point used by orchestration.

AudioSignalProcessor: Extracts speech-related signals from available audio/transcript inputs. It focuses on measurable metrics such as pacing and filler usage. It provides structured outputs for scoring and recommendations. It can later be extended to deeper acoustic features if raw audio is available.

VisionSignalProcessor: Extracts vision-related signals from video/frame inputs. It targets behavioral indicators like attention proxy and facial expression statistics. It can be

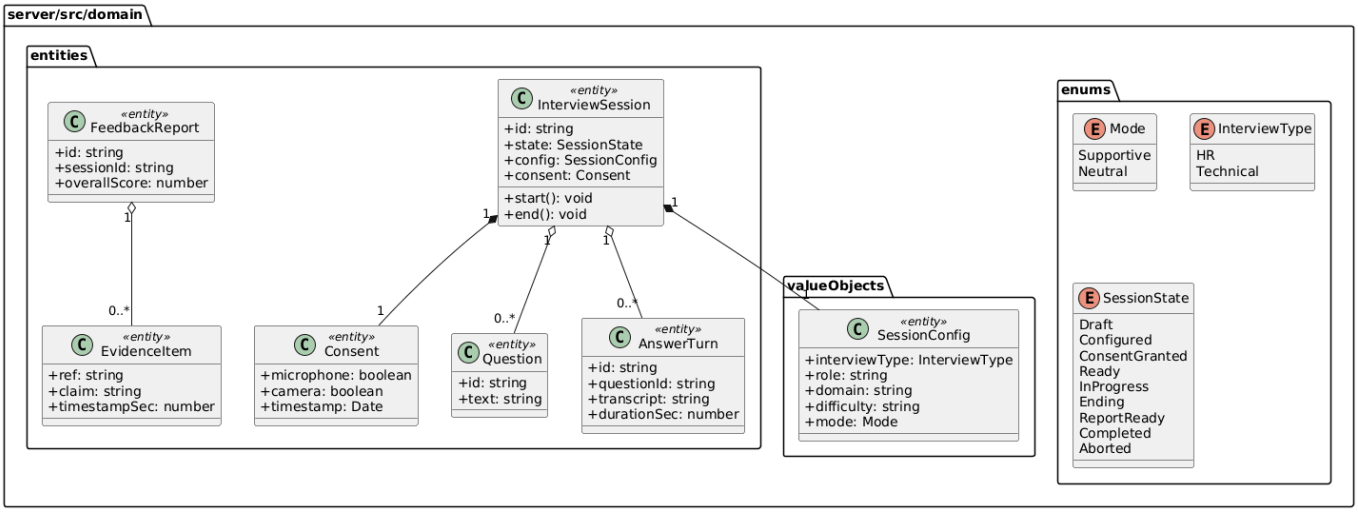
implemented using CPU or GPU inference depending on deployment. It produces structured vision signals for the aggregator.

SignalAggregator: Converts extracted signals into scoring and narrative feedback. It combines audio and vision metrics into a coherent report structure. It is responsible for mapping metrics to recommendations and evidence items. It ensures the final report is consistent and UI-friendly.

AudioSignals: A structured container for audio metrics. It standardizes how audio results are passed across the analysis pipeline. It makes scoring deterministic and testable. It supports future extension with additional audio features.

VisionSignals: A structured container for vision metrics. It standardizes how vision results are passed across the analysis pipeline. It supports confidence/scoring expansion later. It keeps the analysis pipeline decoupled from raw inference outputs. It enables consistent aggregation and reporting.

2.1.5. Domain



2.1.5.1. Enums

Purpose: Provides controlled vocabularies for core business concepts. Enums prevent ambiguous string values and make state transitions and mode selection safer.

Mode: Defines the interview style mode (Supportive vs Neutral). It drives how prompts are created and how the interviewer behaves. It ensures the UI and backend use the same set of allowed values. It simplifies policy enforcement and configuration validation.

InterviewType: Defines the interview category (HR vs Technical). It drives what kind of questions are generated and how performance is interpreted. It helps keep configuration

explicit and consistent. It supports future extension with additional interview types.

SessionState: Defines the lifecycle states of a session. It enables guardrails to enforce correct transitions (e.g., cannot start without consent). It makes orchestration logic easier to reason about and test. It supports clear UI progress display.

2.1.5.2. Value Objects

Purpose: Represents structured domain data that is defined by its values rather than identity. These are commonly embedded into entities and treated as configuration.

SessionConfig: Holds all session setup parameters in one cohesive object. It drives how the interview proceeds and how AI prompts are configured. It ensures a single source of truth for role/domain/difficulty/mode. It is reused across orchestration, AI, and realtime setup.

2.1.5.3. Entities

Purpose: Defines the core business entities with identity and lifecycle. These objects represent the main artifacts of the interview session and its outcomes.

Consent: Represents user permissions for camera and microphone usage. It is essential for enforcing privacy and compliance rules. It supports strict gating before the interview can begin. It can also provide an audit-friendly record of when consent was granted.

InterviewSession: The central entity representing an interview session. It holds the session's identity, state, configuration, and consent. It is the anchor for all actions and transitions during the workflow. It supports consistent state management across the application.

Question: Represents an interview question asked during the session. It provides a structured record of what was asked. It supports traceability between questions and candidate answers. It enables report evidence to reference specific questions.

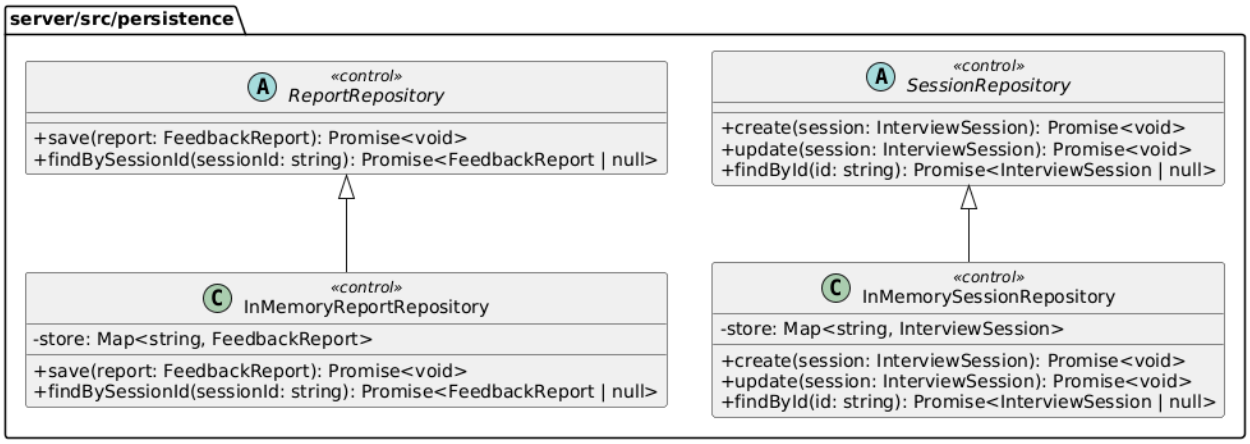
AnswerTurn: Represents a single response turn associated with a question. It stores the transcript and timing-related information needed for analysis. It supports building behavioral metrics and content evaluation. It enables the report to reference specific response segments.

EvidenceItem: Represents a unit of evidence used to justify report feedback. It supports explainability by linking claims to timestamps or references. It keeps reporting structured rather than purely narrative. It enables UI features like "jump to moment" in future.

FeedbackReport: Represents the final outcome artifact of analysis. It summarizes overall

performance and acts as the main output for the feedback dashboard. It can be stored and retrieved independently from the session. It supports future expansion with detailed scoring breakdowns.

2.1.6. Persistence



Purpose: Provides repository interfaces and concrete implementations for storing and retrieving domain artifacts. It isolates storage concerns from orchestration and services.

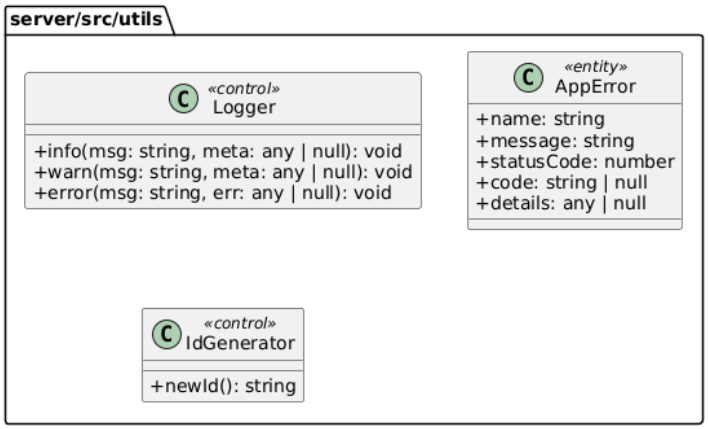
SessionRepository: Defines the contract for persisting and retrieving interview sessions. It keeps orchestration independent of storage implementation. It supports consistent access patterns for session lifecycle changes. It enables easy replacement with a database-backed repository later.

ReportRepository: Defines the contract for persisting and retrieving feedback reports. It standardizes how reports are stored and fetched by session. It enables report retrieval without exposing storage details. It supports future persistence upgrades without changing use-case logic.

InMemorySessionRepository: Provides a simple in-memory implementation for sessions in the MVP. It enables rapid development and testing without database setup. It is suitable for local demos and prototyping. It can be replaced later with a durable storage implementation.

InMemoryReportRepository: Provides a simple in-memory implementation for reports in the MVP. It allows quick iteration on report generation and retrieval. It keeps the MVP deployment lightweight. It can later be replaced with persistent storage without changing orchestration.

2.1.7. Utils

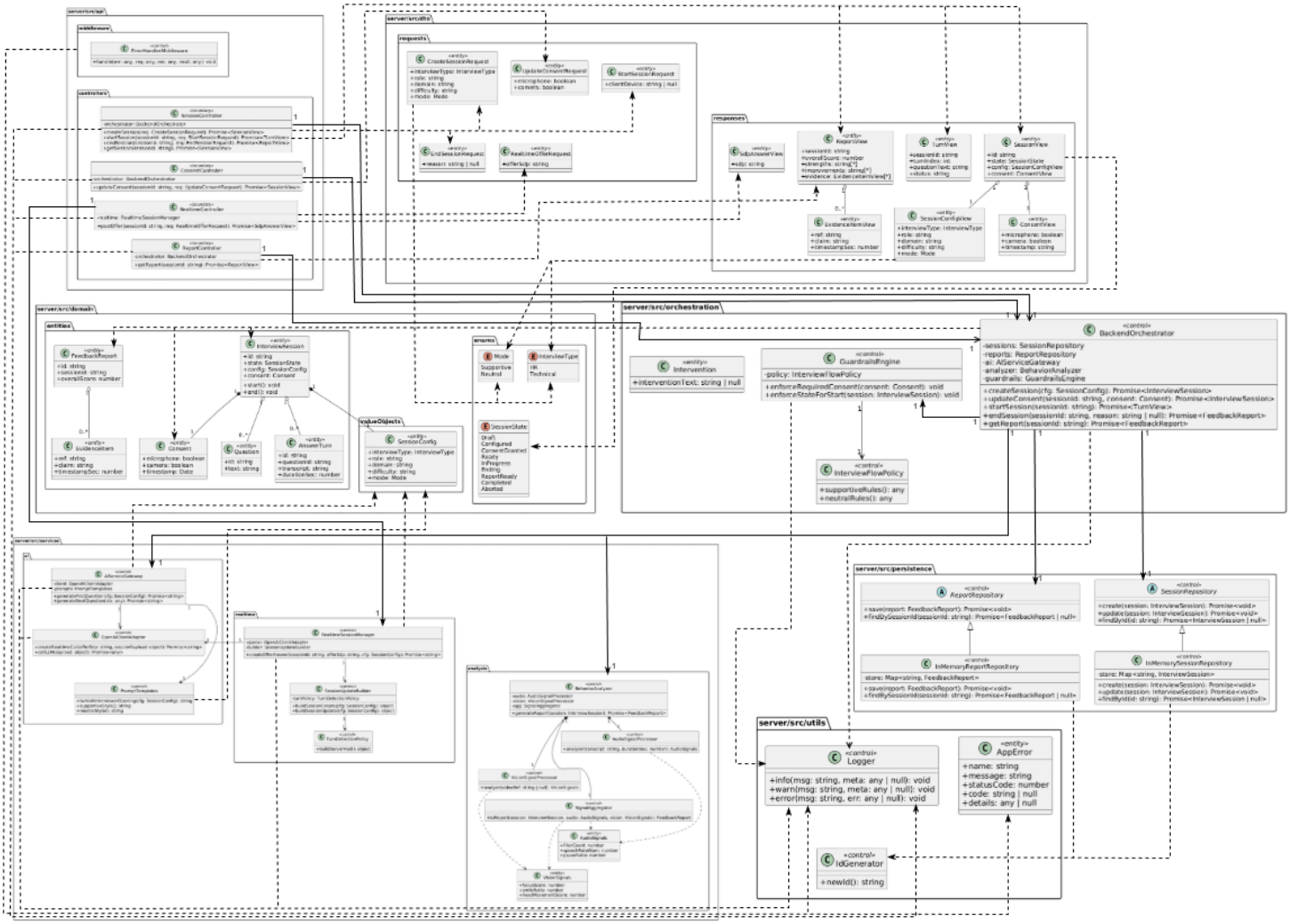


Purpose: Provides shared utility helpers used across layers. It prevents duplication of generic concerns such as IDs, logging, and standardized error handling.

Logger: Centralizes logging so all layers produce consistent log output. It helps debugging and operational monitoring. It can later be extended with log levels and structured logging. It keeps logging concerns out of core business logic.

AppError: Defines a consistent error object format used across the backend. It helps map internal errors to HTTP responses cleanly. It encourages predictable error handling patterns. It reduces ad-hoc error throwing across the codebase.

IdGenerator: Generates unique identifiers for sessions, turns, and reports. It keeps ID generation consistent and testable. It avoids scattering random ID logic across multiple classes. It supports future replacement with UUID libraries or distributed ID strategies.

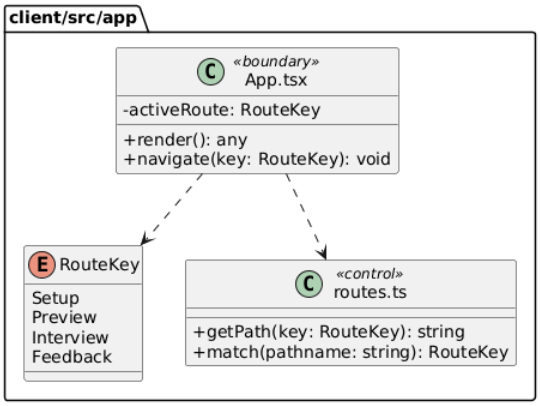


Server Class Diagram

Further investigation: [Server Class Diagram]

2.2. Client

2.2.1. App



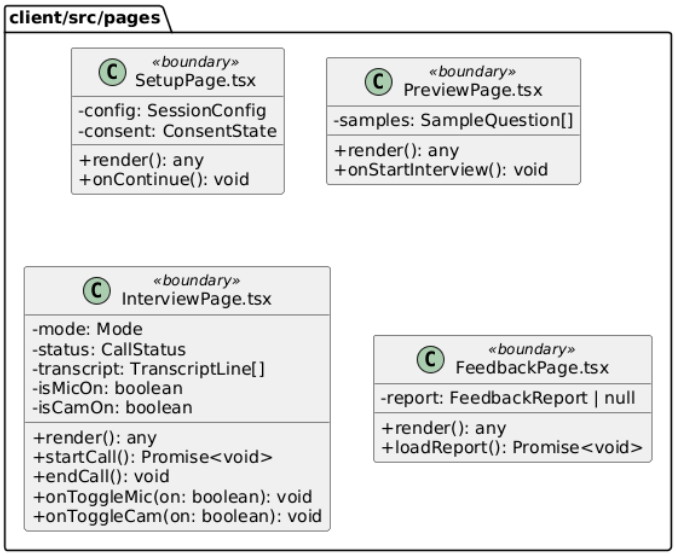
Purpose: Contains the application shell and routing logic. It defines which page is shown for each route and keeps navigation consistent across the UI. This package is the main “composition root” for connecting pages, layout, and global routing rules.

RouteKey: Defines a stable set of route identifiers (Setup, Preview, Interview, Feedback). It prevents hardcoded path strings across the codebase. It makes navigation and route matching type-safe. It also simplifies future route refactors.

Routes (routes.ts): Centralizes route path construction and matching. It maps RouteKey values to concrete URL paths and provides helpers for navigation decisions. It ensures pages do not duplicate route strings. This keeps routing consistent and maintainable.

AppRoot (App.tsx): The top-level React component that wires routing to pages. It decides which page component to render based on the active route. It provides navigation handlers and can host global providers (theme, query clients, etc.). It acts as the UI entry point after main.tsx mounts the application.

2.2.2. Pages



Purpose: Holds route-level UI pages that represent complete screens. Each page composes multiple components, coordinates state for that screen, and calls lib services when needed. Pages should remain thin by delegating reusable UI to components/*.

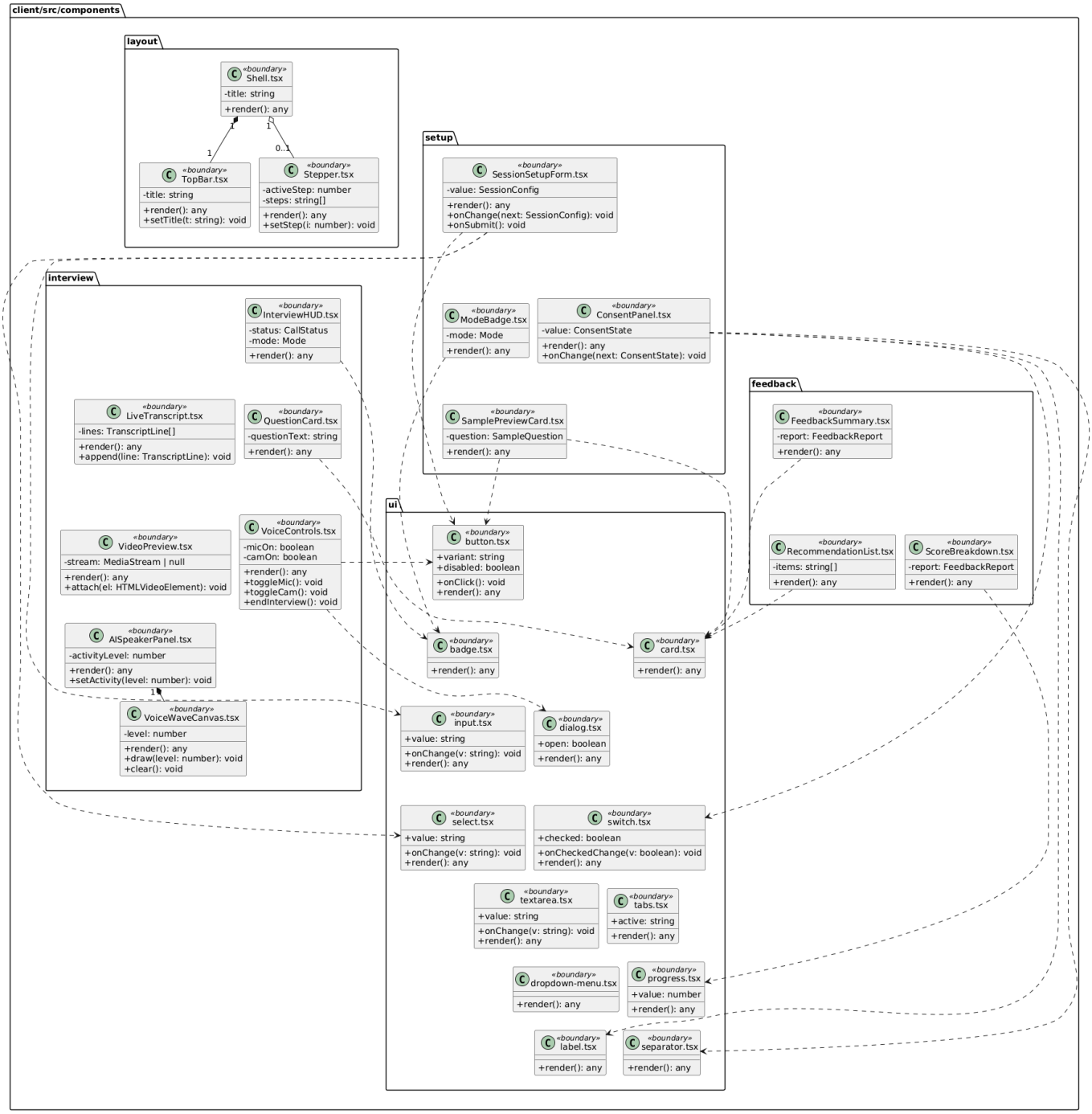
SetupPage: Implements the session configuration step where the user selects interview type/role/domain/difficulty/mode and grants consents. It composes setup components like SessionSetupForm and ConsentPanel. It validates required inputs (especially mandatory camera consent) before continuing. It prepares the session configuration that drives the preview and interview flow.

PreviewPage: Displays a preview of the session setup and shows sample questions after configuration is complete. It renders two sample questions as required and helps the user confirm they are ready. It acts as a checkpoint before entering the live interview screen. It transitions the user into the realtime interview when “Start Interview” is pressed.

InterviewPage: Represents the live interview experience where the user sees their camera preview and the AI speaker activity/voice animation. It manages connection state to the realtime client, transcript updates, and user controls (mic/cam/end). It ensures resources are properly started and stopped (camera/mic capture ends when finishing). It is the core interactive screen of the product.

FeedbackPage: Displays the final report once the interview ends. It loads and renders structured feedback such as overall score, strengths, improvements, and recommendations. It composes feedback-specific components for summaries and breakdowns. It serves as the “results dashboard” of the session.

2.2.3. Components



Purpose: Contains reusable UI building blocks organized by feature and responsibility. Components are grouped by domain (setup/interview/feedback) and shared UI structure (layout/ui). This package enables reuse across pages and keeps pages from becoming monolithic.

2.2.3.1. Layout

Purpose: Provides shared layout primitives used across multiple pages. It standardizes page structure (top bar, step indicators) and creates a consistent look-and-feel across the app. Layout components typically do not contain business logic.

Shell: A page wrapper that provides consistent spacing, background, and top-level composition. It usually hosts TopBar and optionally a Stepper. It ensures each screen has a consistent structure. It makes page-level layouts predictable and reusable.

TopBar: Displays the current screen title and top-level navigation context. It standardizes how headings and primary actions are presented. It helps keep page headers consistent. It can later host global actions (settings, help, logout).

Stepper: Shows progress through the setup-to-feedback flow. It clarifies where the user is in the overall session lifecycle. It helps guide users through multi-step configuration. It supports consistent step definitions across pages.

2.2.3.2. Setup

Purpose: Implements UI components for configuring an interview session. These components collect user inputs, display mode/consent states, and show sample question previews. They keep setup logic modular and testable.

SessionSetupForm: Collects session parameters such as interview type, role, domain, difficulty, and mode. It emits structured changes upward so the page can store a single SessionConfig. It avoids spreading form state across unrelated components. It is the primary configuration UI for the session.

ConsentPanel: Collects and displays consent choices for microphone and camera usage. It supports strict gating rules (camera consent is mandatory) and makes consent explicit. It keeps consent logic separate from general configuration. It ensures privacy-related inputs are not hidden inside other forms.

ModeBadge: Visual indicator for the selected interview mode (Supportive / Neutral). It provides fast recognition of the active mode. It is a presentational component that helps maintain consistent mode branding. It can be reused across pages (preview/interview).

SamplePreviewCard: Displays a single sample interview question in a consistent card layout. It is used by the preview page to show two example questions after setup. It improves user readiness by clarifying the question style. It stays reusable for future “question bank” previews.

2.2.3.3. Interview

Purpose: Contains components used during the live interview screen. These components visualize the AI speaker activity, show the user's camera preview, present the question and transcript, and provide live controls. They are designed to remain reusable and keep InterviewPage readable.

InterviewHUD: Displays session status indicators such as mode, connection state, and recording/transcription status. It provides a consistent "heads-up display" for the live experience. It reduces confusion by surfacing connection state. It can later include timers and turn counters.

QuestionCard: Presents the current interview question prominently. It keeps question display consistent and visually separated from other UI. It supports quick reading in a live interaction. It can later include difficulty tags or question categories.

LiveTranscript: Shows a live-updating transcript of the interview. It helps users see what was captured and supports accessibility. It can be used to debug recognition quality in development. It keeps transcript rendering isolated from connection logic.

VoiceControls: Provides user controls such as microphone toggle, camera toggle, and "end interview." It centralizes control handlers so the page stays clean. It ensures capture/stream stopping is triggered when ending the session. It can later include push-to-talk or mute indicators.

VideoPreview: Renders the user's camera preview (e.g., bottom-left overlay). It handles attaching a media stream to the video element. It keeps media rendering concerns separate from connection logic. It supports consistent styling and positioning across resolutions.

AISpeakerPanel: Represents the "AI is speaking" area of the screen. It visualizes activity level and provides a dedicated space for speaker feedback/animation. It helps replicate the "voice assistant" look you want. It can later show captions, persona info, or speaking indicators.

VoiceWaveCanvas: Draws the animated voice wave based on AI speaking activity. It makes the UI feel responsive by reacting to audio activity levels. It can be reused for both AI and user activity visualization. It is intentionally isolated so animation logic doesn't pollute page components.

2.2.3.4. Feedback

Purpose: Contains components for rendering the final feedback report. These components focus on visualizing scores, recommendations, and summaries. They are reusable pieces for building a dashboard-like feedback page.

FeedbackSummary: Displays the high-level outcome of the report such as overall score and key highlights. It provides an easy first-glance result. It keeps summary formatting consistent across report versions. It can be extended with badges or trend indicators.

ScoreBreakdown: Visualizes detailed scoring aspects and sub-metrics. It helps users understand what contributed to the overall score. It encourages actionable interpretation of results. It can later map scores to evidence timestamps.

RecommendationList: Displays concrete improvement suggestions in a clear list format. It turns analysis output into actionable next steps. It keeps recommendation styling consistent. It can later group items by category (communication, confidence, clarity).

2.2.3.5. UI

Purpose: Provides shared UI primitives (buttons, inputs, cards, dialogs, etc.) used by feature components. These primitives keep styling consistent and reduce duplicated UI code. They are typically generic and do not contain business logic.

UIButton (button.tsx): A reusable button primitive that standardizes variants, disabled states, and click handling. It ensures consistent interaction styling across the app. It simplifies accessibility and focus styling. It reduces repetitive button markup.

UICard (card.tsx): A container primitive for consistent panel/card layout. It standardizes padding, borders, and section spacing. It is used for previews, question cards, and feedback panels. It keeps visual hierarchy consistent.

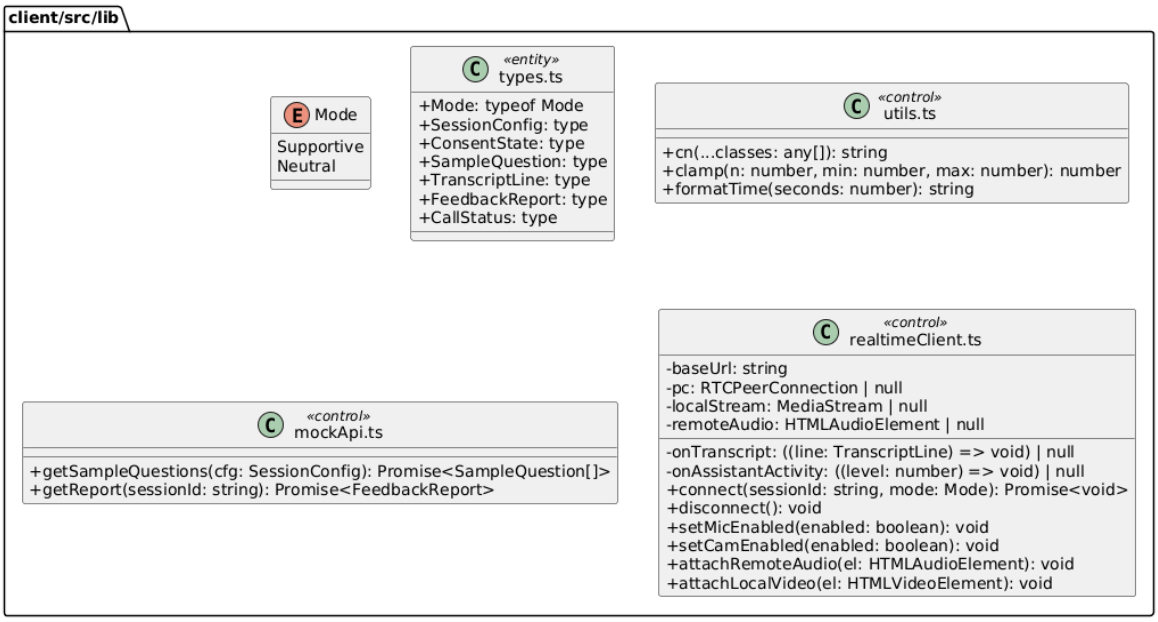
UIBadge (badge.tsx): A small label primitive used for mode tags and status indicators. It provides consistent badge sizing and styling. It's useful for compact metadata display. It helps highlight key states at a glance.

UIDialog (dialog.tsx): A modal dialog primitive for confirmations and overlays. It centralizes open/close behavior patterns. It supports consistent modal UX across features. It is typically used for "end interview" confirmations.

UIInput / UISelect / UISwitch / UITextarea: Standardized form controls for consistent input behavior and styling. They reduce repeated markup and help keep forms predictable. They support accessibility patterns and consistent labels. They are used heavily in session setup and consent UI.

UITabs / UIProgress / UIDropdownMenu / UILabel / UISeparator: Supporting primitives for structuring UI sections, progress indicators, menus, labeling, and visual separation. They make complex layouts easier to build cleanly. They keep styling consistent across pages. They allow easy iteration on design without touching feature logic.

2.2.4. Lib



Purpose: Holds shared client-side logic that is not purely UI. It contains types, utilities, API adapters, and realtime connection logic. This package keeps networking and helper code out of React components to improve maintainability and testing.

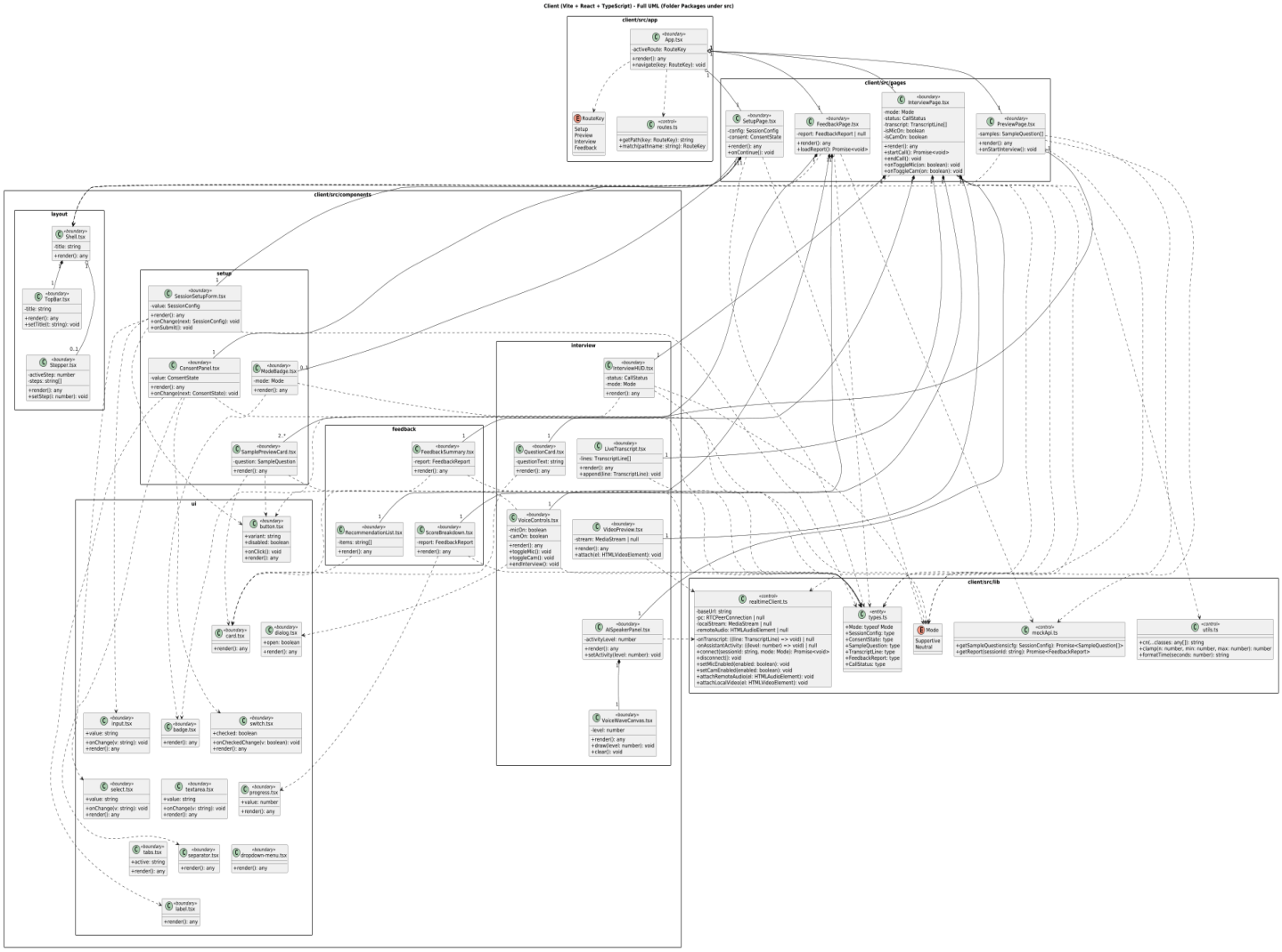
TypesModule (types.ts): Central location for shared TypeScript types and enums used across pages and components. It ensures consistent contracts for configuration, consent state, transcript lines, and report structures. It prevents duplicated ad-hoc type definitions. It makes refactoring safer by updating types in one place.

Utils (utils.ts): Contains small reusable helper functions such as class name merging (cn) and formatting helpers. It avoids duplicating formatting logic across components. It keeps UI code cleaner and more readable. It provides consistent formatting behavior across the app.

MockApi (mockApi.ts): Provides a lightweight API layer for demo and local development. It can return sample questions and mock reports without requiring the backend. It helps build UI independently from server readiness. It can later be replaced or wrapped by real API calls.

RealtimeClient (realtimeClient.ts): Encapsulates realtime interview connectivity, including WebRTC connection setup and media stream handling. It exposes high-level methods like

connect/disconnect and mic/cam toggles. It forwards transcript events and AI activity levels back to the UI via callbacks. It keeps complex realtime logic out of React components for clarity and reuse.



Client Class Diagram

Further investigation: [Client Class Diagram]

3. Class Interfaces

3.1. User

3.1.1. API

3.1.1.1. Controllers

Class: SessionController

Description: Handles session lifecycle endpoints such as creating, starting, ending, and retrieving sessions. It converts request DTOs into application-level calls and returns response DTOs. It keeps business logic out of the HTTP layer by delegating to the orchestrator. It is the primary entry point for the setup and session flow in the UI.

Attributes

Visibility	Name	Type
private	orchestrator	BackendOrchestrator

Methods

Visibility	Signature	Return Type
public	createSession(req: CreateSessionRequest)	Promise<SessionView>
public	startSession(sessionId: string, req: StartSessionRequest)	Promise<TurnView>
public	endSession(sessionId: string, req: EndSessionRequest)	Promise<ReportView>
public	getSession(sessionId: string)	Promise<SessionView>

Class: ConsentController

Description: Manages consent updates for microphone and camera. It ensures consent changes go through the same use-case rules as the rest of the system. This keeps consent gating consistent and auditable. It supports mandatory-consent-before-start requirements.

Attributes

Visibility	Name	Type

private	orchestrator	BackendOrchestrator

Methods

Visibility	Signature	Return Type
public	updateConsent(sessionId: string, req: UpdateConsentRequest)	Promise<SessionView>

Class: RealtimeController

Description: Handles realtime signaling endpoints (offer/answer exchange). It separates WebRTC negotiation from normal session CRUD operations. It forwards SDP offers to a realtime manager and returns an SDP answer. This keeps realtime logic isolated and maintainable.

Attributes

Visibility	Name	Type
private	realtime	RealtimeSessionManager

Methods

Visibility	Signature	Return Type
public	postOffer(sessionId: string, req: RealtimeOfferRequest)	Promise<SdpAnswerView>

Class: ReportController

Description: Provides a read endpoint for fetching the feedback report. It keeps reporting separate from session lifecycle endpoints. It delegates report retrieval to the orchestrator to avoid duplicating business logic. It returns a UI-friendly report representation.

Attributes

Visibility	Name	Type
private	orchestrator	BackendOrchestrator

Methods

Visibility	Signature	Return Type
public	getReport(sessionId: string)	Promise<ReportView>

3.1.1.2. Middleware

Class: ErrorHandlerMiddleware

Description: Centralizes error handling for all API endpoints. It normalizes errors into consistent HTTP responses and prevents leaking sensitive details. It reduces repeated try/catch in controllers. It can also attach request IDs and logging metadata.

Attributes: (none)

Methods

Visibility	Signature	Return Type
public	handle(err: any, req: any, res: any, next: any)	void

3.1.2. DTO

3.1.2.1. Requests

Class: CreateSessionRequest

Description: Represents the request payload for creating a new interview session. It captures configuration choices that affect question generation and session behavior. It acts as a stable contract between frontend and backend. It is converted into a domain SessionConfig by orchestration.

Attributes

Visibility	Name	Type
public	interviewType	InterviewType
public	role	string
public	domain	string

public	difficulty	string
public	mode	Mode

Methods

Visibility	Signature	Return Type
(none)	*(DTO data carrier)*	*(none)*

Class: UpdateConsentRequest

Description: Represents the request payload for updating consent settings. It carries microphone and camera permissions from the client. It enables strict consent gating before session start. It is mapped into the domain Consent object.

Attributes

Visibility	Name	Type
public	microphone	boolean
public	camera	boolean

Methods: (none)

Class: StartSessionRequest

Description: Represents the request payload for starting a session. It can include optional client metadata such as device/browser information. It keeps “start” concerns separate from “create session” concerns. It allows future start-time parameters without breaking the create contract.

Attributes

Visibility	Name	Type
public	clientDevice	string | null

Methods: (none)

Class: EndSessionRequest

Description: Represents the request payload for ending a session. It may include a reason for termination such as user action or timeout. It supports consistent end-of-session handling and reporting. It is used to trigger report generation in orchestration.

Attributes

Visibility	Name	Type
public	reason	string | null

Methods: (none)

Class: RealtimeOfferRequest

Description: Represents the realtime signaling offer sent by the browser. It carries the SDP offer needed to establish the WebRTC connection. It isolates realtime negotiation from other session endpoints. It enables straightforward validation and extension.

Attributes

Visibility	Name	Type
public	offerSdp	string

Methods: (none)

3.1.2.2. Responses

Class: SessionView

Description: Client-facing representation of a session snapshot. It includes the session ID, lifecycle state, configuration, and consent. It is returned after creation and consent updates so the UI can render the current step. It is intentionally shaped for frontend use rather than internal domain structure.

Attributes

Visibility	Name	Type

public	id	string
public	state	SessionState
public	config	SessionConfigView
public	consent	ConsentView

Methods: (none)

Class: TurnView

Description: Client-facing view of an interview turn. It provides the question text and turn status for the live interview UI. It includes a turn index to support progress display. It is used when the session starts or when a new question is produced.

Attributes

Visibility	Name	Type
public	sessionId	string
public	turnIndex	int
public	questionText	string
public	status	string

Methods: (none)

Class: ReportView

Description: Client-facing feedback report displayed on the dashboard. It contains summary scores, strengths, and improvement suggestions. It also carries evidence items to increase explainability. It is returned after session end or via the report endpoint.

Attributes

Visibility	Name	Type
public	sessionId	string

public	overallScore	number
public	strengths	string[*]
public	improvements	string[*]
public	evidence	EvidenceItemView[*]

Methods: *(none)*

Class: SdpAnswerView

Description: Response wrapper containing the SDP answer returned by the backend. The client uses it to complete the WebRTC handshake. It keeps the realtime response contract minimal and clear. It can be extended later with ICE server hints if needed.

Attributes

Visibility	Name	Type
public	sdp	string

Methods: (none)

Class: SessionConfigView

Description: Response-friendly version of session configuration. It is included in SessionView so the UI can show selected settings. It references enums for mode and interview type to keep values consistent. It decouples frontend display from internal domain representation.

Attributes

Visibility	Name	Type
public	interviewType	InterviewType
public	role	string
public	domain	string

public	difficulty	string
public	mode	Mode

Methods: (none)

Class: ConsentView

Description: Response-friendly version of consent state. It includes microphone and camera values plus a serialized timestamp for UI display. It avoids exposing raw Date objects in API responses. It is embedded inside SessionView.

Attributes

Visibility	Name	Type
public	microphone	boolean
public	camera	boolean
public	timestamp	string

Methods: (none)

Class: EvidenceItemView

Description: Response representation of a single evidence item. It provides a reference label, a claim, and a timestamp in seconds. It supports explainable feedback and future “jump to moment” features. It is included inside ReportView.

Attributes

Visibility	Name	Type
public	ref	string
public	claim	string
public	timestampSec	number

Methods: (none)

3.1.3. Orchestration

Class: BackendOrchestrator

Description: Implements the main use-case workflows for session creation, consent updates, starting, ending, and report retrieval. It coordinates repositories, services, and guardrails to keep business flow consistent. It is the single point that enforces the correct sequence of actions. It keeps controllers thin and testable.

Attributes

Visibility	Name	Type
private	sessions	SessionRepository
private	reports	ReportRepository
private	ai	AIServiceGateway
private	analyzer	BehaviorAnalyzer
private	guardrails	GuardrailsEngine

Methods

Visibility	Signature	Return Type
public	createSession(cfg: SessionConfig)	Promise<InterviewSession>
public	updateConsent(sessionId: string, consent: Consent)	Promise<InterviewSession>
public	startSession(sessionId: string)	Promise<TurnView>
public	endSession(sessionId: string, reason: string | null)	Promise<FeedbackReport>
public	getReport(sessionId: string)	Promise<FeedbackReport>

Class: GuardrailsEngine

Description: Enforces critical constraints such as mandatory consent and valid state transitions. It consults a policy object for mode-specific rules. It prevents invalid session flows from proceeding. It centralizes safety and correctness checks across the backend.

Attributes

Visibility	Name	Type
private	policy	InterviewFlowPolicy

Methods

Visibility	Signature	Return Type
public	enforceRequiredConsent(consent: Consent)	void
public	enforceStateForStart(session: InterviewSession)	void

Class: InterviewFlowPolicy

Description: Encapsulates the high-level rules that define supportive and neutral interview behavior. It separates policy definition from enforcement logic. It allows the system to evolve behavior rules without changing orchestrator code. It provides a consistent source of “mode rules”.

Attributes: *(none)*

Methods

Visibility	Signature	Return Type
public	supportiveRules()	any
public	neutralRules()	any

Class: Intervention

Description: Represents a structured intervention output such as a warning or corrective message. It can be used when guardrails detect a rule violation. It provides a consistent object to pass intervention information across layers. It can be expanded later for richer moderation actions.

Attributes

Visibility	Name	Type
public	interventionText	string | null

Methods: (none)

3.1.4. Services

3.1.4.1. AI

Class: AIServiceGateway

Description: Provides high-level AI operations used by the orchestration layer. It generates interviewer questions and ensures mode-appropriate style via prompt templates. It hides provider details by delegating to a client adapter. It keeps AI concerns centralized and consistent.

Attributes

Visibility	Name	Type
private	client	OpenAIClientAdapter
private	prompts	PromptTemplates

Methods

Visibility	Signature	Return Type
public	generateFirstQuestion(cfg: SessionConfig)	Promise<string>
public	generateNextQuestion(ctx: any)	Promise<string>

Class: OpenAIClientAdapter

Description: Wraps communication with the OpenAI APIs and isolates SDK details. It provides stable internal methods used by both AI and realtime modules. It centralizes logging and error translation for provider calls. It reduces coupling to vendor-specific request formats.

Attributes: (none)

Methods

Visibility	Signature	Return Type
public	createRealtimeCall(offerSdp: string, sessionPayload: object)	Promise<string>
public	callLLM(payload: object)	Promise<any>

Class: PromptTemplates

Description: Stores reusable prompt templates for consistent interviewer behavior. It ensures Turkish interviewer opening and mode-specific styles are applied uniformly. It helps keep prompts maintainable and testable. It allows changing tone without editing orchestration logic.

Attributes: *(none)*

Methods

Visibility	Signature	Return Type
public	turkishInterviewerOpening(cfg: SessionConfig)	string
public	supportiveStyle()	string
public	neutralStyle()	string

3.1.4.2. Realtime

Class: RealtimeSessionManager

Description: Coordinates realtime session negotiation using the client's SDP offer. It builds provider session payloads through a builder and delegates provider calls to the adapter. It returns the SDP answer needed by the client to establish the connection. It keeps realtime protocol logic modular and isolated.

Attributes

Visibility	Name	Type

private	openai	OpenAIClientAdapter
private	builder	SessionUpdateBuilder

Methods

Visibility	Signature	Return Type
public	createOfferAnswer(sessionId: string, offerSdp: string, cfg: SessionConfig)	Promise<string>

Class: SessionUpdateBuilder

Description: Builds provider-specific session creation and update payload objects. It isolates schema construction from the session manager. It makes realtime payload changes easier when provider parameters evolve. It keeps configuration logic reusable and consistent across calls.

Attributes

Visibility	Name	Type
private	turnPolicy	TurnDetectionPolicy

Methods

Visibility	Signature	Return Type
public	buildSessionCreate(cfg: SessionConfig)	object
public	buildSessionUpdate(cfg: SessionConfig)	object

Class: TurnDetectionPolicy

Description: Encapsulates turn detection configuration such as server-side VAD policies. It prevents the realtime builder from hardcoding detection settings. It allows changing detection behavior in one place. It helps adapt to provider schema updates more safely.

Attributes: (none)

Methods

Visibility	Signature	Return Type
public	buildServerVad()	object

3.1.4.3. Analysis

Class: BehaviorAnalyzer

Description: Orchestrates post-session behavior analysis. It delegates audio and vision feature extraction to dedicated processors and uses an aggregator to produce the final report. It keeps analysis modular and testable. It is called by orchestration when a session ends.

Attributes

Visibility	Name	Type
private	audio	AudioSignalProcessor
private	vision	VisionSignalProcessor
private	agg	SignalAggregator

Methods

Visibility	Signature	Return Type
public	generateReport(session: InterviewSession)	Promise<FeedbackReport>

Class: AudioSignalProcessor

Description: Extracts speech-related signals from transcript and timing data. It computes measurable metrics used for scoring and recommendations. It outputs structured results for aggregation. It can be extended later with deeper acoustic features if raw audio is available.

Attributes: *(none)*

Methods

Visibility	Signature	Return Type

public	analyze(transcript: string, durationSec: number)	AudioSignals

Class: VisionSignalProcessor

Description: Extracts vision-related signals from available video references. It targets attention proxies and basic facial/head movement metrics. It outputs structured results for aggregation. It can later use CPU or GPU inference without changing upstream orchestration.

Attributes: (none)

Methods

Visibility	Signature	Return Type
public	analyze(videoRef: string | null)	VisionSignals

Class: SignalAggregator

Description: Combines extracted audio and vision signals into a single report artifact. It applies scoring logic and maps signals to strengths and improvements. It keeps reporting consistent and UI-friendly. It is the central point for converting metrics into feedback output.

Attributes: (none)

Methods

Visibility	Signature	Return Type
public	toReport(session: InterviewSession, audio: AudioSignals, vision: VisionSignals)	FeedbackReport

Class: AudioSignals

Description: Holds structured audio metrics produced by the audio processor. It standardizes how audio results are passed to the aggregator. It supports deterministic scoring and unit testing. It is designed to be extended with more metrics later.

Attributes

Visibility	Name	Type
public	fillerCount	number
public	speechRateWpm	number
public	pauseRatio	number

Methods: (none)

Class: VisionSignals

Description: Holds structured vision metrics produced by the vision processor. It standardizes vision results passed to the aggregator. It supports clear weighting/aggregation logic. It is designed to be extended with more vision metrics later.

Attributes

Visibility	Name	Type
public	focusScore	number
public	smileRatio	number
public	headMovementScore	number

Methods: (none)

3.1.5. Domain

3.1.5.1. Enums

Enum: Mode

Description: Defines the interviewer tone and behavioral style. It drives prompt construction and guardrail policy selection. It prevents invalid string values by constraining mode choices. It is shared across request/response DTOs and domain config.

Values: Supportive, Neutral

Enum: InterviewType

Description: Defines the interview category. It influences what kind of questions and evaluation style the system uses. It keeps configuration explicit and consistent. It can be extended with more interview types later.

Values: HR, Technical

Enum: SessionState

Description: Defines the lifecycle states for an interview session. It enables guardrails to prevent invalid transitions. It supports predictable UI step rendering and backend workflow sequencing. It also improves auditability of session progression.

Values: Draft, Configured, ConsentGranted, Ready, InProgress, Ending, ReportReady, Completed, Aborted

3.1.5.2. Value Objects

Class: SessionConfig

Description: Holds the session configuration parameters as a single cohesive object. It is used throughout orchestration and services to ensure consistent behavior. It reduces parameter passing complexity by grouping related settings. It also supports mode-based prompt and realtime configuration.

Attributes

Visibility	Name	Type
public	interviewType	InterviewType
public	role	string
public	domain	string
public	difficulty	string
public	mode	Mode

Methods: (none)

3.1.5.3. Entities

Class: Consent

Description: Represents the user's permissions for microphone and camera usage. It enables strict consent gating before starting the interview. It also provides a timestamp for auditability and UI display. It is stored as part of the session state.

Attributes

Visibility	Name	Type
public	microphone	boolean
public	camera	boolean
public	timestamp	Date

Methods: (none)

Class: InterviewSession

Description: The central domain entity for an interview session. It stores identity, current lifecycle state, configuration, and consent information. It aggregates asked questions and user answer turns for later reporting. It provides state transition operations such as starting and ending the session.

Attributes

Visibility	Name	Type
public	id	string
public	state	SessionState
public	config	SessionConfig
public	consent	Consent

Methods

Visibility	Signature	Return Type
public	start()	void

public	end()	void

Class: Question

Description: Represents an interview question asked to the candidate. It provides a stable structure for storing the content of questions. It supports linking questions to turns and evidence in the report. It helps maintain traceability across the interview flow.

Attributes

Visibility	Name	Type
public	id	string
public	text	string

Methods: (none)

Class: AnswerTurn

Description: Represents one candidate response turn. It links to a question and stores transcript and response duration. It provides the main input for post-session behavior analysis. It supports timing-based metrics and evidence references.

Attributes

Visibility	Name	Type
public	id	string
public	questionId	string
public	transcript	string
public	durationSec	number

Methods: (none)

Class: EvidenceItem

Description: Represents a piece of evidence included in feedback. It references a moment and provides a claim describing an observed behavior. It supports explainable reports and potential playback features. It keeps evidence structured and consistent.

Attributes

Visibility	Name	Type
public	ref	string
public	claim	string
public	timestampSec	number

Methods: (none)

Class: FeedbackReport

Description: Represents the final report artifact produced after the session ends. It stores overall scoring information and is linked to a specific session. It can be persisted and retrieved independently. It serves as the main output for the feedback UI.

Attributes

Visibility	Name	Type
public	id	string
public	sessionId	string
public	overallScore	number

Methods: (none)

3.1.6. Persistence

Abstract Class: SessionRepository

Description: Defines the storage contract for interview sessions. It abstracts away the underlying database or storage mechanism. Orchestration depends on this interface to load and persist session state transitions. It enables easy swapping of persistence implementations later.

Attributes: (none)

Methods

Visibility	Signature	Return Type
public	create(session: InterviewSession)	Promise<void>
public	update(session: InterviewSession)	Promise<void>
public	findById(id: string)	Promise<InterviewSession | null>

Abstract Class: ReportRepository

Description: Defines the storage contract for feedback reports. It allows storing and fetching reports independently from sessions. It provides consistent access patterns for the report UI. It keeps orchestration independent of storage implementation details.

Attributes: (none)

Methods

Visibility	Signature	Return Type
public	save(report: FeedbackReport)	Promise<void>
public	findBySessionId(sessionId: string)	Promise<FeedbackReport | null>

Class: InMemorySessionRepository

Description: Provides a lightweight in-memory session store for MVP development. It supports fast iteration and local testing without a database. It implements the repository contract using a map keyed by session ID. It can later be replaced with a real database-backed repository.

Attributes

Visibility	Name	Type
private	store	Map<string, InterviewSession>

Methods

Visibility	Signature	Return Type
public	create(session: InterviewSession)	Promise<void>
public	update(session: InterviewSession)	Promise<void>
public	findById(id: string)	Promise<InterviewSession | null>

Class: InMemoryReportRepository

Description: Provides a lightweight in-memory report store for MVP development. It enables quick report generation experiments without database setup. It implements the repository contract using a map keyed by session ID. It can later be replaced with persistent storage.

Attributes

Visibility	Name	Type
private	store	Map<string, FeedbackReport>

Methods

Visibility	Signature	Return Type
public	save(report: FeedbackReport)	Promise<void>
public	findBySessionId(sessionId: string)	Promise<FeedbackReport | null>

3.1.7. Utils

Class: Logger

Description: Central logging utility used across layers. It provides consistent log formatting and log levels. It reduces duplicated console usage in controllers and services. It can later be extended to structured logging and external log sinks.

Attributes: (none)

Methods

Visibility	Signature	Return Type
public	info(msg: string, meta: any | null)	void
public	warn(msg: string, meta: any | null)	void
public	error(msg: string, err: any | null)	void

Class: AppError

Description: Standard application error model. It carries an HTTP-friendly status code and an optional internal error code. It can include structured details for debugging while still controlling what is returned to the client. It enables consistent error handling in middleware and orchestration.

Attributes

Visibility	Name	Type
public	name	string
public	message	string
public	statusCode	number
public	code	string | null
public	details	any | null

Methods: (none)

Class: IdGenerator

Description: Generates unique IDs for domain artifacts like sessions and reports. It keeps ID creation consistent and testable. It avoids scattering random ID logic across multiple classes. It can later be replaced with UUID libraries without changing business code.

Attributes: (none)

Methods

Visibility	Signature	Return Type

public	newId()	string

3.2. Client

3.2.1. App

Class: Routes (routes.ts)

Description: Centralizes route path definitions and mapping logic. It converts a route key into a concrete URL path and can map a pathname back to a route key. This prevents hardcoded strings scattered across pages. It makes navigation more maintainable and type-safe.

Attributes: *(none)*

Methods

Visibility	Signature	Return Type
public	getPath(key: RouteKey)	string
public	match(pathname: string)	RouteKey

Class: AppRoot (App.tsx)

Description: Acts as the root component that composes pages and routing. It decides which route-level page to render based on the active route. It provides navigation functions so the UI can move between setup, preview, interview, and feedback. It is the main composition point for global UI structure.

Attributes

Visibility	Name	Type
private	activeRoute	RouteKey

Methods

Visibility	Signature	Return Type

public	render()	any
public	navigate(key: RouteKey)	void

3.2.2. Pages

Class: SetupPage (SetupPage.tsx)

Description: Implements the session configuration step where the user selects role/domain/difficulty/mode and grants consent. It composes setup components and validates required fields before allowing the user to continue. It produces a single configuration object used by later screens. It enforces mandatory camera consent before progressing.

Attributes

Visibility	Name	Type
private	config	SessionConfig
private	consent	ConsentState

Methods

Visibility	Signature	Return Type
public	render()	any
public	onContinue()	void

Class: PreviewPage (PreviewPage.tsx)

Description: Shows a confirmation screen after setup and displays sample questions. It renders exactly two sample questions to preview question style before starting the interview. It provides a clear transition into the live interview experience. It can fetch sample questions via a lightweight API module.

Attributes

Visibility	Name	Type
private	samples	SampleQuestion[]

Methods

Visibility	Signature	Return Type
public	render()	any
public	onStartInterview()	void

Class: InterviewPage (InterviewPage.tsx)

Description: Represents the live interview screen, combining camera preview, AI voice activity animation, question display, and live transcript. It coordinates the realtime connection lifecycle and toggles for microphone/camera. It updates UI state based on transcript events and assistant activity. It is responsible for stopping media capture and disconnecting when the interview ends.

Attributes

Visibility	Name	Type
private	mode	Mode
private	status	CallStatus
private	transcript	TranscriptLine[]
private	isMicOn	boolean
private	isCamOn	boolean

Methods

Visibility	Signature	Return Type
public	render()	any
public	startCall()	Promise<void>
public	endCall()	void
public	onToggleMic(on: boolean)	void
public	onToggleCam(on: boolean)	void

Class: FeedbackPage (FeedbackPage.tsx)

Description: Displays the feedback report produced after the interview ends. It loads the report data and composes feedback components for summary, score breakdown, and recommendations. It keeps report rendering separated from interview flow logic. It can work with mock or real API sources.

Attributes

Visibility	Name	Type
private	report	FeedbackReport | null

Methods

Visibility	Signature	Return Type
public	render()	any
public	loadReport()	Promise<void>

3.2.3. Components

3.2.3.1. Layout

Class: Shell (Shell.tsx)

Description: Provides a shared layout wrapper used by multiple pages. It standardizes page spacing, background, and top-level structure. It typically hosts the top bar and optional stepper to keep navigation consistent. It helps pages remain focused on feature content rather than layout.

Attributes

Visibility	Name	Type
private	title	string

Methods

Visibility	Signature	Return Type
public	render()	any

Class: TopBar (TopBar.tsx)

Description: Displays the screen title and top-level context for the current page. It enforces consistent header styling across the application. It can later host global actions like settings or help. It keeps header behavior centralized and reusable.

Attributes

Visibility	Name	Type
private	title	string

Methods

Visibility	Signature	Return Type
public	render()	any
public	setTitle(t: string)	void

Class: Stepper (Stepper.tsx)

Description: Shows progress across the multi-step interview flow. It improves user orientation by indicating where the user is in setup/preview/interview/feedback. It supports consistent step naming and ordering. It can be reused across multiple pages when needed.

Attributes

Visibility	Name	Type
private	activeStep	number
private	steps	string[]

Methods

Visibility	Signature	Return Type
public	render()	any

public	setStep(i: number)	void

3.2.3.2. Setup

Class: SessionSetupForm (SessionSetupForm.tsx)

Description: Collects the session configuration (type/role/domain/difficulty/mode) in a structured form. It emits changes as a single config object so the page can store state cleanly. It avoids scattered state management across multiple controls. It is designed for validation and easy extension.

Attributes

Visibility	Name	Type
private	value	SessionConfig

Methods

Visibility	Signature	Return Type
public	render()	any
public	onChange(next: SessionConfig)	void
public	onSubmit()	void

Class: ConsentPanel (ConsentPanel.tsx)

Description: Collects and displays user consent for microphone and camera. It keeps privacy-related inputs explicit and separate from general setup. It supports strict gating by making required permissions easy to validate. It can display warnings when mandatory consent is missing.

Attributes

Visibility	Name	Type
private	value	ConsentState

Methods

Visibility	Signature	Return Type
public	render()	any
public	onChange(next: ConsentState)	void

Class: ModeBadge (ModeBadge.tsx)

Description: Renders a compact visual label for the selected interview mode. It helps users quickly confirm whether the session is supportive or neutral. It keeps mode styling consistent across screens. It is purely presentational and reusable.

Attributes

Visibility	Name	Type
private	mode	Mode

Methods

Visibility	Signature	Return Type
public	render()	any

Class: SamplePreviewCard (SamplePreviewCard.tsx)

Description: Displays a sample question in a consistent card layout. It is used during preview to show two example questions after setup. It improves user readiness by clarifying question style and tone. It is reusable for future question previews.

Attributes

Visibility	Name	Type
private	question	SampleQuestion

Methods

Visibility	Signature	Return Type

public	render()	any

3.2.3.3. Interview

Class: InterviewHUD (InterviewHUD.tsx)

Description: Displays real-time session indicators such as connection status and active mode. It provides a stable “HUD” region so the live interview experience remains understandable. It reduces confusion during reconnects or permission issues. It can later include timers and turn counters.

Attributes

Visibility	Name	Type
private	status	CallStatus
private	mode	Mode

Methods

Visibility	Signature	Return Type
public	render()	any

Class: QuestionCard (QuestionCard.tsx)

Description: Presents the current question in a readable, focused UI element. It isolates question formatting from the page and other components. It keeps visual hierarchy consistent across interview turns. It can later show tags like difficulty or question category.

Attributes

Visibility	Name	Type
private	questionText	string

Methods

Visibility	Signature	Return Type

public	render()	any

Class: LiveTranscript (LiveTranscript.tsx)

Description: Renders a live transcript stream of the interview. It improves transparency by showing what the system captured. It supports accessibility and debugging of speech-to-text quality. It remains reusable so transcript rendering is not tied to a single page.

Attributes

Visibility	Name	Type
private	lines	TranscriptLine[]

Methods

Visibility	Signature	Return Type
public	render()	any
public	append(line: TranscriptLine)	void

Class: VoiceControls (VoiceControls.tsx)

Description: Exposes user controls for microphone and camera toggles and ending the interview. It centralizes live controls so the main page stays clean. It ensures capture can be stopped and the session ended intentionally. It can also host confirmation dialogs to prevent accidental end.

Attributes

Visibility	Name	Type
private	micOn	boolean
private	camOn	boolean

Methods

Visibility	Signature	Return Type
public	render()	any
public	toggleMic()	void
public	toggleCam()	void
public	endInterview()	void

Class: VideoPreview (VideoPreview.tsx)

Description: Displays the user's camera preview, typically as a small overlay in the corner. It is responsible for attaching a media stream to a video element safely. It keeps media rendering separated from connection logic. It enables consistent positioning and responsive scaling.

Attributes

Visibility	Name	Type
private	stream	MediaStream | null

Methods

Visibility	Signature	Return Type
public	render()	any
public	attach(el: HTMLVideoElement)	void

Class: AISpeakerPanel (AISpeakerPanel.tsx)

Description: Represents the AI speaker area of the live interview UI. It visualizes when the interviewer is speaking using an activity level. It provides a dedicated space for the voice wave animation. It is designed to mimic a "voice assistant" look without requiring an avatar.

Attributes

Visibility	Name	Type

private	activityLevel	number

Methods

Visibility	Signature	Return Type
public	render()	any
public	setActivity(level: number)	void

Class: VoiceWaveCanvas (VoiceWaveCanvas.tsx)

Description: Draws an animated voice waveform based on a numeric activity level. It makes the UI feel reactive to AI speech output. It isolates animation/drawing code so it does not clutter page logic. It can be reused for different wave styles or additional speakers later.

Attributes

Visibility	Name	Type
private	level	number

Methods

Visibility	Signature	Return Type
public	render()	any
public	draw(level: number)	void
public	clear()	void

3.2.3.4. Interview

Class: FeedbackSummary (FeedbackSummary.tsx)

Description: Displays the top-level result of the feedback report. It surfaces the overall score and key highlights at a glance. It keeps summary UI consistent and reusable. It can be extended with badges and evidence links later.

Attributes

Visibility	Name	Type
private	report	FeedbackReport

Methods

Visibility	Signature	Return Type
public	render()	any

Class: ScoreBreakdown (ScoreBreakdown.tsx)

Description: Visualizes a more detailed breakdown of scoring factors. It helps users understand why the overall score looks the way it does. It keeps chart/progress UI logic isolated from pages. It is designed for easy extension when new metrics are added.

Attributes

Visibility	Name	Type
private	report	FeedbackReport

Methods

Visibility	Signature	Return Type
public	render()	any

Class: RecommendationList (RecommendationList.tsx)

Description: Displays improvement suggestions as a structured list. It turns analysis output into actionable items the user can follow. It keeps recommendation styling consistent across the app. It can later group recommendations by category or severity.

Attributes

Visibility	Name	Type

private	items	string[]

Methods

Visibility	Signature	Return Type
public	render()	any

3.2.3.5. UI

Class: UIButton (button.tsx)

Description: A reusable button primitive that standardizes interactions across the UI. It supports variants and disabled state handling. It reduces duplicated button markup in feature components. It can encapsulate accessibility and consistent focus styling.

Attributes

Visibility	Name	Type
public	variant	string
public	disabled	boolean

Methods

Visibility	Signature	Return Type
public	onClick()	void
public	render()	any

Class: UICard (card.tsx)

Description: A container primitive for consistent card layouts. It standardizes padding, borders, and structure for panels. It is used for previews, questions, and feedback sections. It keeps visual hierarchy consistent throughout the app.

Attributes: (none)

Methods

Visibility	Signature	Return Type
public	render()	any

Class: UIBadge (badge.tsx)

Description: A small label primitive used for mode/status badges. It keeps badge sizing and styling consistent across screens. It helps highlight important metadata at a glance. It remains generic and reusable.

Attributes: (none)

Methods

Visibility	Signature	Return Type
public	render()	any

Class: UIDialog (dialog.tsx)

Description: A modal dialog primitive for confirmations and overlays. It centralizes open/close behavior so features remain consistent. It helps prevent accidental destructive actions by enabling confirmation flows. It can be reused in multiple screens.

Attributes

Visibility	Name	Type
public	open	boolean

Methods

Visibility	Signature	Return Type
public	render()	any

Class: UIInput (input.tsx)

Description: A standardized text input control. It provides consistent styling and change handling. It reduces repeated input markup and supports predictable validation wiring. It is heavily used in configuration forms.

Attributes

Visibility	Name	Type
public	value	string

Methods

Visibility	Signature	Return Type
public	onChange(v: string)	void
public	render()	any

Class: UISelect (select.tsx)

Description: A standardized select/dropdown control. It keeps selection UI consistent across forms. It ensures a predictable change signature for parent components. It can later support grouped options or async options.

Attributes

Visibility	Name	Type
public	value	string

Methods

Visibility	Signature	Return Type
public	onChange(v: string)	void
public	render()	any

Class: UISwitch (switch.tsx)

Description: A standardized boolean toggle control. It is used for consent toggles and other on/off settings. It keeps switch behavior consistent and easy to validate. It supports clean wiring to state updates.

Attributes

Visibility	Name	Type
public	checked	boolean

Methods

Visibility	Signature	Return Type
public	onCheckedChange (v: boolean)	void
public	render()	any

Class: UITextarea (textarea.tsx)

Description: A standardized multi-line text input control. It ensures consistent styling for longer inputs. It provides a predictable change handler signature. It can be reused for notes, feedback text, or longer form inputs.

Attributes

Visibility	Name	Type
public	value	string

Methods

Visibility	Signature	Return Type
public	onChange(v: string)	void
public	render()	any

Class: UITabs (tabs.tsx)

Description: A tab navigation primitive for switching between content panes. It keeps tab UI consistent and reusable. It can simplify complex screens by organizing sections. It supports an active tab value contract.

Attributes

Visibility	Name	Type
public	active	string

Methods

Visibility	Signature	Return Type
public	render()	any

Class: UIProgress (progress.tsx)

Description: A progress/percentage indicator primitive. It is useful for score breakdowns, loading indicators, or step progress visuals. It standardizes how progress is represented across the UI. It can later support animations or labels.

Attributes

Visibility	Name	Type
public	value	number

Methods

Visibility	Signature	Return Type
public	render()	any

Class: UIDropdownMenu (dropdown-menu.tsx)

Description: A reusable dropdown menu primitive for compact action menus. It standardizes menu interactions and accessibility patterns. It helps keep layouts clean by hiding secondary actions. It can be used across pages when needed.

Attributes: (none)

Methods

Visibility	Signature	Return Type
public	render()	any

Class: UILabel (label.tsx)

Description: A small primitive for consistent form labeling. It ensures labels look uniform across the app. It supports accessible form patterns by pairing labels with inputs. It reduces repeated styling code.

Attributes: (none)

Methods

Visibility	Signature	Return Type
public	render()	any

Class: UISeparator (separator.tsx)

Description: A visual separator primitive used to divide sections. It standardizes spacing and border styles between UI blocks. It improves readability in complex screens. It remains generic and reusable across all features.

Attributes: (none)

Methods

Visibility	Signature	Return Type
public	render()	any

3.2.4. Lib

Class: TypesModule (types.ts)

Description: Provides shared TypeScript types and enums used across the UI. It keeps contracts consistent between pages and components (session config, consent, transcript lines, feedback report). It reduces duplication and prevents mismatched shapes across the codebase. It enables safer refactoring by centralizing type ownership.

Attributes

Visibility	Name	Type
public	Mode	typeof Mode

public	SessionConfig	type
public	ConsentState	type
public	SampleQuestion	type
public	TranscriptLine	type
public	FeedbackReport	type
public	CallStatus	type

Methods: (none)

Class: Utils (utils.ts)

Description: Contains small, reusable utility helpers for UI logic. It centralizes class name composition and common formatting helpers. It prevents repetitive code across components and improves consistency. It is intentionally lightweight and framework-agnostic.

Attributes: *(none)*

Methods

Visibility	Signature	Return Type
public	cn(...classes: any[])	string
public	clamp(n: number, min: number, max: number)	number
public	formatTime(seconds: number)	string

Class: MockApi (mockApi.ts)

Description: Provides a simple mock data layer for UI development and demos. It returns sample questions and mock reports without requiring a running backend. It helps build and test the UI workflow early. It can later be swapped with real API calls behind the same function signatures.

Attributes: (none)

Methods

Visibility	Signature	Return Type
public	getSampleQuestions(cfg: SessionConfig)	Promise<SampleQuestion[]>
public	getReport(sessionId: string)	Promise<FeedbackReport>

Class: RealtimeClient (realtimeClient.ts)

Description: Encapsulates the realtime interview connection logic and media handling. It manages WebRTC state, local media capture, and remote audio playback. It exposes a stable API to pages/components for connect/disconnect and mic/cam toggles. It publishes transcript and assistant activity updates through callback hooks to keep UI reactive.

Attributes

Visibility	Name	Type
private	baseUrl	string
private	pc	RTCPeerConnection | null
private	localStream	MediaStream | null
private	remoteAudio	HTMLAudioElement | null
private	onTranscript	((line: TranscriptLine) => void) | null
private	onAssistantActivity	((level: number) => void) | null

Methods

Visibility	Signature	Return Type
public	connect(sessionId: string, mode: Mode)	Promise<void>

public	disconnect()	void
public	setMicEnabled(enabled: boolean)	void
public	setCamEnabled(enabled: boolean)	void
public	attachRemoteAudio(el: HTMLAudioElement)	void
public	attachLocalVideo(el: HTMLVideoElement)	void

4. Glossary

- **Mock Interview:** A simulated interview session designed to provide structured and realistic practice conditions.
- **HR Interview:** A behavioral interview type focusing on communication skills, motivation, and cultural fit.
- **Technical Interview (Non-Coding):** A knowledge-based technical interview that does not require live coding tasks.
- **Mode (Supportive / Neutral):** Defines interviewer tone and behavioral style. Supportive mode provides guidance and clarification, while Neutral mode maintains a formal evaluation-oriented tone.
- **Session Lifecycle (SessionState):** The predefined progression of session states (e.g., Draft, InProgress, Completed) that governs workflow transitions.
- **Guardrails:** Rule-based constraints that enforce valid session flow, required consent, and policy compliance.
- **Backend Orchestrator:** The application-layer component responsible for coordinating services, repositories, and guardrail enforcement.
- **Turn:** A single interaction unit consisting of one interviewer question and one candidate response.
- **Transcript:** The textual representation of spoken responses captured during the session.
- **Behavioral Signals:** Quantifiable metrics derived from audio or video analysis (e.g., filler usage, speech rate, focus score).
- **Evidence Item:** A structured reference linking feedback observations to specific timestamps.
- **Feedback Report:** The final structured output generated after session completion, including scores, strengths, and improvement suggestions.
- **Repository:** An abstraction layer responsible for storing and retrieving domain entities.

5. References

[1] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston, MA, USA: Prentice Hall, 2017.

[2] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2002.

[3] ISO/IEC/IEEE 42010:2011, *Systems and Software Engineering — Architecture Description*, ISO, 2011.

[4] ISO/IEC/IEEE 29148:2018, *Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*, ISO, 2018.

[5] IEEE, *Ethically Aligned Design: A Vision for Prioritizing Human Well-being with Autonomous and Intelligent Systems*, 1st ed., IEEE, 2019.

[6] National Institute of Standards and Technology (NIST), *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1, 2023.

[7] A. Bai et al., “Constitutional AI: Harmlessness from AI Feedback,” arXiv:2212.08073, 2022.

[8] OpenAI, “Best Practices for Prompt Engineering,” 2023.

[9] IETF, “WebRTC 1.0: Real-Time Communication Between Browsers,” W3C Recommendation, 2021.

[10] M. Handley, V. Jacobson, and C. Perkins, “SDP: Session Description Protocol,” RFC 4566, IETF, 2006.

[11] P. Ekman, “An Argument for Basic Emotions,” *Cognition and Emotion*, vol. 6, no. 3–4, pp. 169–200, 1992.

[12] B. W. Schuller et al., “Speech Emotion Recognition: Two Decades in a Nutshell, Benchmarks, and Ongoing Trends,” *Communications of the ACM*, vol. 61, no. 5, pp. 90–99, 2018.

[13] A. Mehrabian, *Silent Messages: Implicit Communication of Emotions and Attitudes*. Belmont, CA, USA: Wadsworth, 1971.

[14] F. Doshi-Velez and B. Kim, “Towards a Rigorous Science of Interpretable Machine Learning,” arXiv:1702.08608, 2017.

[15] M. T. Ribeiro, S. Singh, and C. Guestrin, “Why Should I Trust You? Explaining the Predictions of Any Classifier,” in *Proc. ACM SIGKDD*, 2016, pp. 1135–1144.

[16] A. Cavoukian, “Privacy by Design: The 7 Foundational Principles,” Information and Privacy Commissioner of Ontario, 2009.

[17] European Union, *General Data Protection Regulation (GDPR)*, Regulation (EU) 2016/679, 2016.

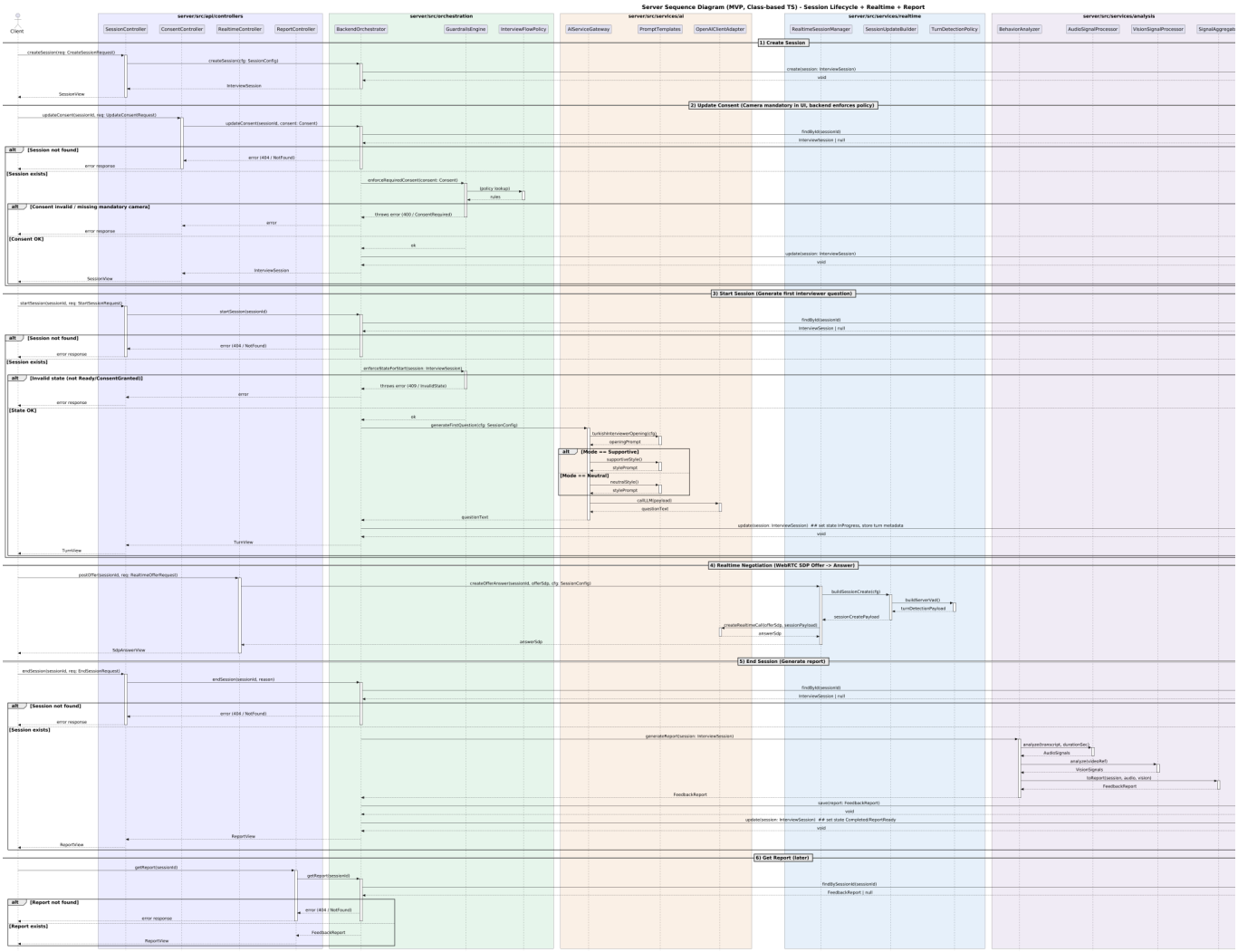
6. Appendix

Appendix A: Tracebility Matrix

Requirement ID	HLD Component	LLD Package/Class	Notes
FR-01	Web Client Setup	SetupPage, SessionSetupForm	Interview type selection
FR-02	Web Client Setup	SessionConfig, CreateSessionRequest	Role + company/industry
FR-03	Web Client Setup	SessionConfig	Domain + difficulty
FR-04	Interview Orchestrator	InterviewFlowPolicy, PromptTemplates	Mode-specific behavior
FR-05	Live Interview	RealtimeSessionManager, InterviewPage	10–15 minute sessions
FR-06	STT Pipeline	AIServiceGateway, OpenAIClientAdapter	Speech-to-text
FR-07	LLM Questioning	AIServiceGateway, PromptTemplates	Question generation
FR-08	Guardrails	GuardrailsEngine.enforceTimeLimit	Time-bound prompting
FR-09	Supportive Mode	InterviewFlowPolicy.supportiveRules	Off-topic redirection
FR-10	Supportive Mode	InterviewFlowPolicy.supportiveRules	Uncertainty assistance

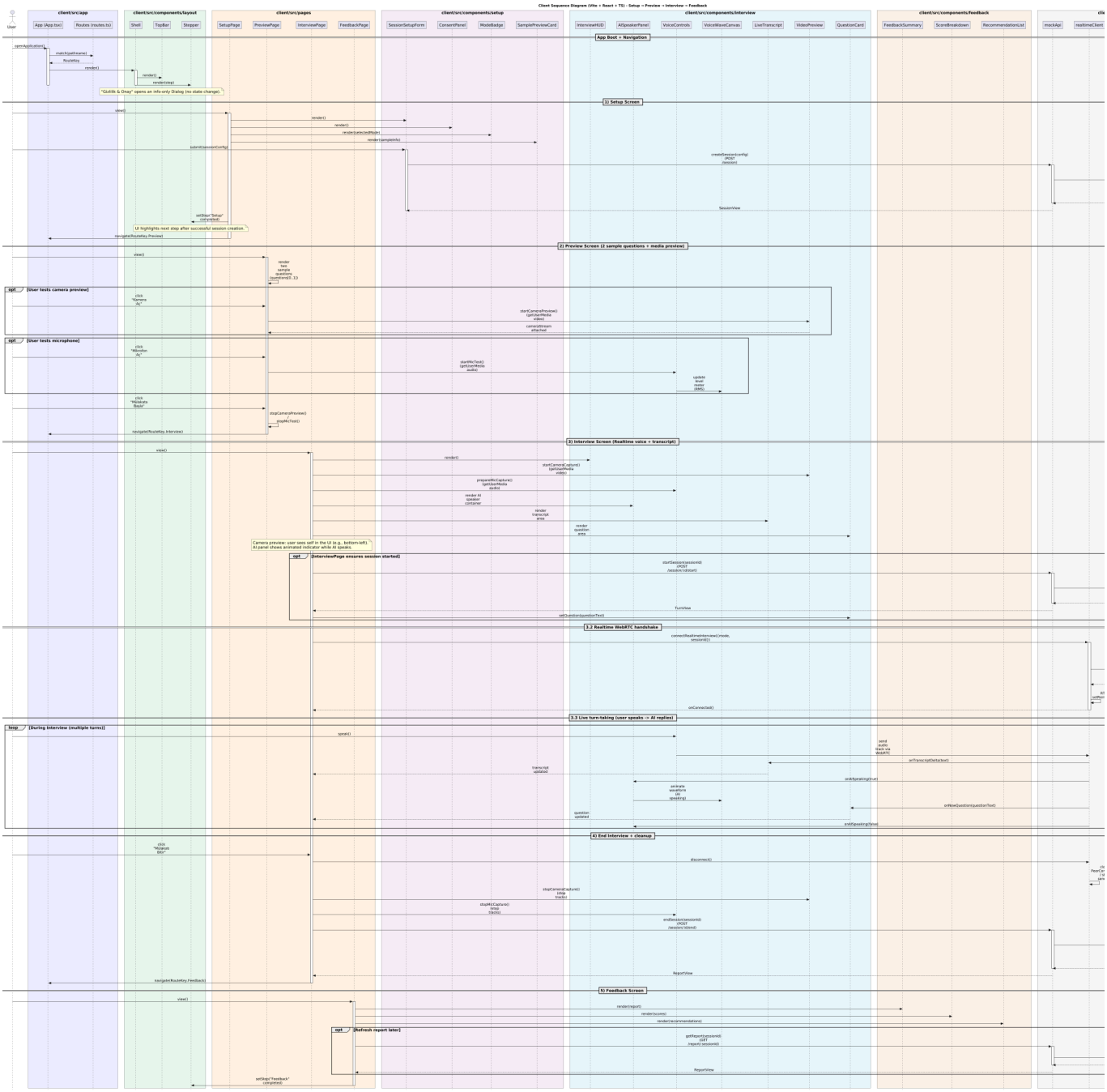
FR-11	Feedback Generation	BehaviorAnalyzer, SignalAggregator	Post-session report
FR-12	TTS Pipeline	OpenAIClientAdapter	Interviewer voice output
FR-13	Vision Analysis	VisionSignalProcessor	Camera-based signals
NFR-01	Web Client UX	SetupPage, Stepper	Guided flow
NFR-02	Realtime (Live)	RealtimeSessionManager	Responsiveness
NFR-03	Resilience	BehaviorAnalyzer	Graceful degradation
NFR-04	Security	Backend API	API keys server-side
NFR-05	Modularity	api/services/orchestration	Extensibility

Appendix B: Server Sequence Diagram



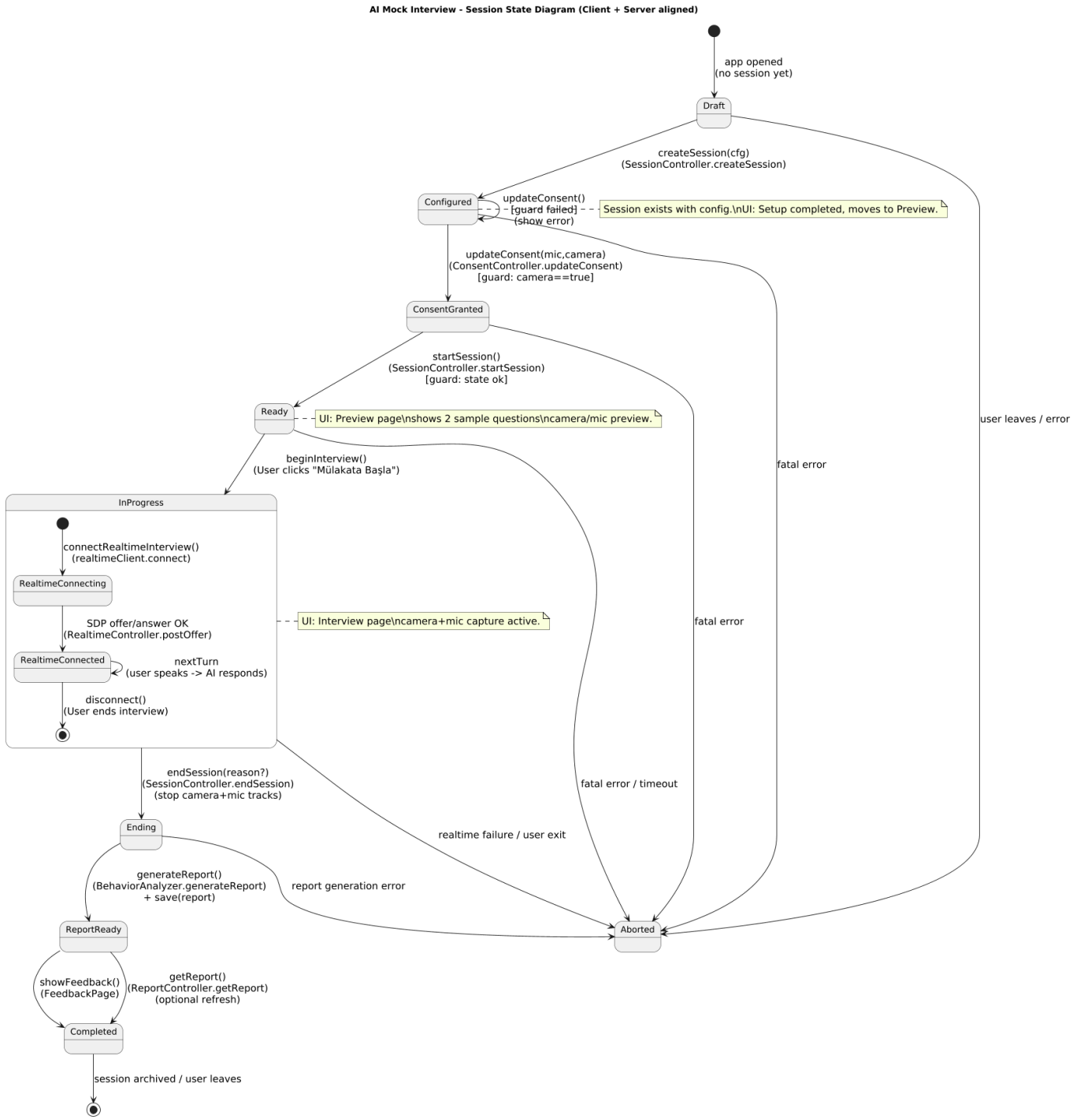
Further investigation: [Server Sequence Diagram]

Appendix C: Client Sequence Diagram



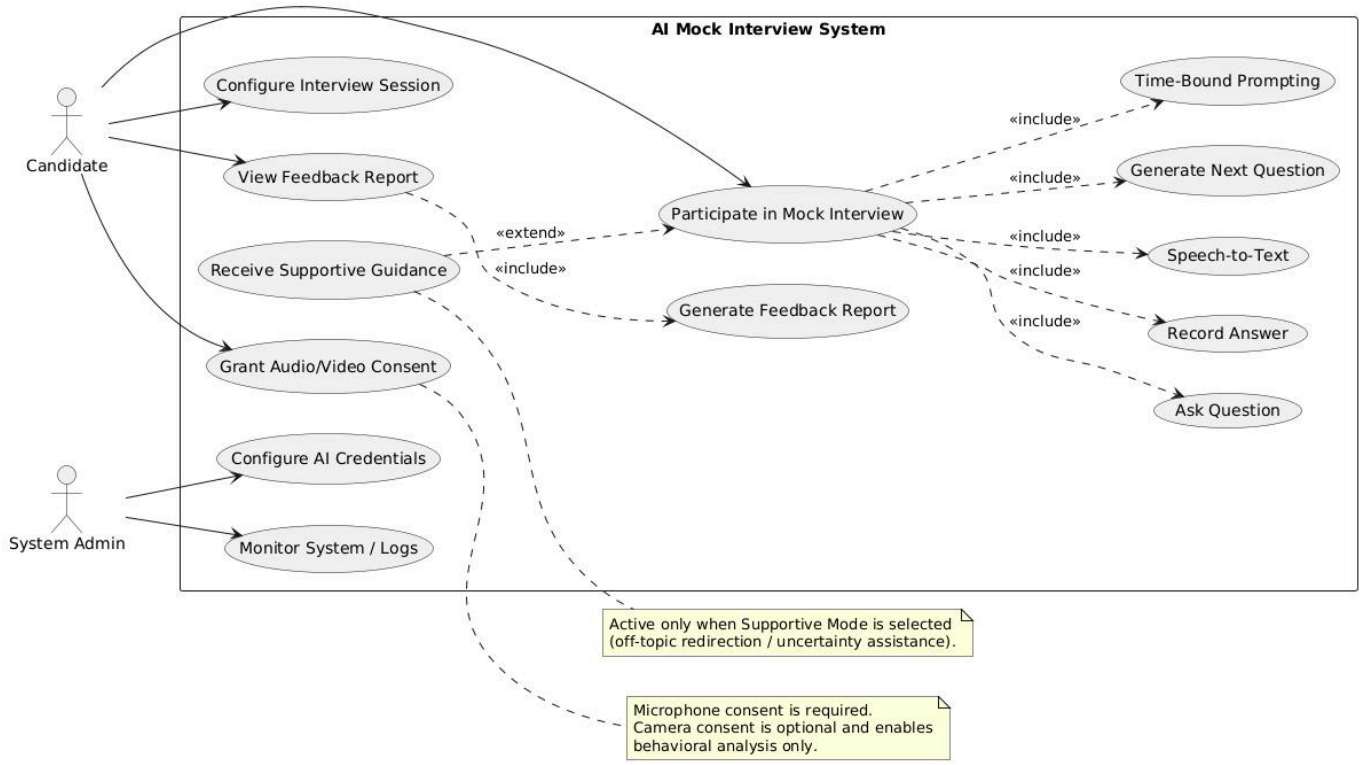
Further investigation: [Client Sequence Diagram]

Appendix D: State Diagram



Further investigation: [State Diagram]

Appendix E: Use Case Diagram



Further investigation: [Use Case Diagram]